

THE UNIVERSITY OF SYDNEY

The University of Sydney

ELEC5620 Stage 1 Report

Members:

Chuanyang Wang

Ziyi Wang

Angelina

Joshua Bates

An Assignment submitted for the UoS:

ELEC5620

October 12, 2026

1 Project Requirements

1.1 Project Purpose and Scope

BakeFlow AI is an AI-assisted back-office system for a small home-based bakery. It supports an owner who manages customer enquiries, orders, inventory, production, finances and marketing alongside baking. The project aims to reduce administrative effort while maintaining consistent records and human control over important decisions.

The system serves two main users: the Customer, who submits order requirements, reviews quotations and requests updates; and the Human Owner, who manages business information, reviews results and approves decisions. Four specialist AI agents support these activities, coordinated by a deterministic Workflow Service.

1.2 Core Functional Requirements

The system shall capture custom order requirements, clarify missing information, and support order tracking, changes and cancellation. Uncertain allergen information and ingredient-affecting customisations shall be referred to the owner. Inventory and production functions shall calculate recipe-based ingredient requirements, check available stock, generate shortage-only shopping lists, and propose schedules that meet stock, capacity and completion-time constraints.

Pricing and bookkeeping functions shall calculate quotations from material, labour, packaging and additional costs using the owner's target profit margin. They shall also record valid income and expenses, reject duplicate transactions and calculate profitability. Marketing functions shall use business summaries to support campaign planning and generate promotional content for validation and owner approval before export.

The Workflow Service shall coordinate these functions through explicit rules and record important transitions and decisions. Orders shall be confirmed only after customers accept owner-approved quotations, and production shall require approval of the current schedule version.

1.3 Quality Requirements and Constraints

Calculations and business-record updates shall use deterministic services. AI outputs shall be validated before they affect stored data. Failed or invalid AI operations shall be retried at most twice before failure is reported or referred to the owner.

The system shall authenticate users, restrict protected operations to authorised users and retain audit information for business changes and approvals. Changes to approved quotations, production plans or marketing content shall require renewed approval; routine task-progress updates shall not invalidate an approved production plan. Repeated processing shall not duplicate financial entries or stock deductions; ingredients shall be deducted once per order after all required production tasks are complete.

The prototype assumes browser-based interfaces, a single application host, a shared database and an external LLM accessed through a server-side gateway. Physical baking,

real payment execution, ingredient purchasing and public content publication remain outside the automated workflow.

2 Ad Hoc Requirements

2.1 Sales and Customer Service

AR-01: Sales and customer service support. The system should help customers describe custom bakery orders and obtain reliable information about their orders. The Sales Agent should interpret natural-language requests and identify the requested products, quantities, flavours, themes, required collection time, allergen concerns and any stated budget. It should ask for missing or ambiguous information rather than invent details, and allow the customer to review the interpreted requirements.

Ingredient-affecting customisations and uncertain allergen questions should be referred to the owner before the order proceeds to inventory checking or quotation. Deterministic services should validate the information before storing or updating an order. Confirming these details should create a validated draft, not a confirmed sale; formal order confirmation requires customer acceptance of an owner-approved quotation.

The system should also allow customers to track their own orders and request changes or cancellation. Such requests should be checked against access permissions and the current order state, with changes affecting an existing quotation or production plan returned for appropriate checking and renewed approval. Refunds and changes affecting work already in production should be referred to the owner. AI output should not independently approve decisions, alter shared recipes or overwrite validated business records.

2.2 Inventory and Production

AR-02: Inventory and production support. The system should help the bakery owner understand ingredient availability and prepare a feasible production schedule. It should calculate ingredient requirements from confirmed product recipes and order quantities, compare these requirements with available stock, and produce a shopping list containing only shortages. The Inventory and Production Agent should explain shortages and suggest a production plan, while deterministic services perform the quantity calculations and validate the proposed plan against daily capacity and delivery times. The owner must approve the current version of the schedule before it can be used for production. The system should record received ingredients, maintain task progress, and deduct an order's ingredients once after its production tasks are complete. AI output must not independently change stock, approve a schedule, or overwrite validated business records.

Modelling assumptions and execution rules. The following rules clarify how the existing classes are used in this section; they do not introduce additional classes or attributes.

IP-01: Completion and deduction. An order is production-complete only when all of its required production tasks are completed. Its ingredient quantities are deducted once, not once per task. The existing record store must retain a durable record of this deduction so repeated completion processing has no further effect. The stock change and its completion/deduction record must succeed together; a failed update must not be reported as completed.

IP-02: Stock shared between orders. Stock checks subtract the requirements of other active, approved orders whose ingredients have not yet been deducted. These commitments are derived from existing order and schedule records, not separate reservation objects. Each order is counted once even if it has several tasks. When a schedule is revised, its previous version is replaced in this calculation rather than counted again. Availability and capacity are rechecked in the guarded save so two approvals cannot both consume the same available stock.

IP-03: Recipe resolution. The owner reviews ingredient-changing requests and selects a suitable product and recipe before inventory or costing calculations. Unsupported requests return for clarification. Free text must not silently alter a shared recipe or an existing order's calculated requirements. Recipe and quantity changes affecting a committed order require renewed checking of the affected plan.

IP-04: Versioned approval. Changing planned task times, adding or removing tasks, or changing their orders creates a new schedule version and invalidates the previous active approval. Historical decisions remain in the audit records. Task progress updates do not by themselves revise the approved plan. Rescheduling requires review before affected work continues.

IP-05: Capacity and states. The daily capacity attribute represents available production work time per day for the bakery's single production team. Planned task intervals must fit within that daily allowance, not overlap for that team, and meet the order's required time. An approvable schedule contains at least one task. Task status strings use `Planned`, `InProgress`, `Blocked`, and `Completed`; these are allowed values of the existing attribute, not a new enumeration class.

2.3 Pricing and Bookkeeping

AR-03: Pricing and bookkeeping support. The system should help the bakery owner price custom orders and understand business profitability. It should calculate material, labour, packaging and additional costs, then apply the owner's target profit margin to prepare a quotation. These calculations should use deterministic software rather than an LLM. The Pricing and Bookkeeping Agent should explain the calculated price, while the owner approves the final quotation before it is presented to the customer. The system should also record income and expenses, reject invalid or duplicate transactions, and calculate profit by order and product. The agent should explain these financial results using the recorded data.

2.4 Marketing

AR-04: Data-driven marketing support. The system should use recorded sales, profit, inventory and production-capacity data to recommend marketing campaigns and to generate promotional content for the bakery. The Marketing Agent should identify which products are worth promoting, taking account of seasonal events, current stock levels, profitability and the production capacity available in the campaign period, and should then draft social media content and product descriptions for those products.

Product names, prices and availability referenced in generated content should be validated against stored business records rather than produced by the language model alone. The owner must approve the campaign plan before dependent content is generated, and must approve the current content version before manual export or publication. Editing approved content, including content already in the calendar, requires renewed validation and approval. The system should not publish automatically.

3 Ad Hoc Overall Software Design

BakeFlow AI is an AI-powered back-office system for a home-based custom bakery. Figure 1 gives an informal overview of its components, conceptual layers and main calls before the formal models in later sections.

The interface layer contains the customer interface and owner dashboard. The control layer contains the deterministic `WorkflowService`, which routes work according to predefined states and rules, coordinates validation, records transitions and pauses for required owner decisions. `AccessControl` and `ApprovalService` represent authorisation and approval responsibilities. The logic layer contains four specialist AI agents and the deterministic business services. The data layer contains `ValidatedRecordStore` and the audit log. These layers separate responsibilities; they do not require every call to pass through each adjacent layer.

`WorkflowService` calls agents and services directly. Agents interpret, plan, explain and propose content; services calculate and validate business results. Agents return proposals and cannot commit business changes themselves. Workflow, business and approval services access the shared store as shown. The store holds validated drafts as well as approved records, so saving a draft is distinct from authorising a high-impact action. Quotations, production schedules, campaign plans and public content require approval of the current record version before the corresponding action may proceed. Relevant customer acceptance and other use-case checks still apply.

`LLMGateway` is a server-side client used by the agents, not the external language model itself. It communicates over HTTPS with the external LLM provider, which has no direct access to business records. Exact prices, stock quantities and profitability remain the responsibility of deterministic services, while the workflow rules determine what happens next.

The diagram is informal and does not replace the class, package or deployment models in Sections 8, 13 and 14. `AccessControl` is a capability provided by `UserAccount` and `ValidatedRecordStore.authorise()`, not a separate class. `IAgentTask` labels the common task contract represented by `AIAgent.execute` in the class model. `ApprovalService` is shown in the control layer because it gates progress, while the package diagram groups it with shared model and services because all business modules use its records. The arrows show representative calls and data access rather than every operation in the system.

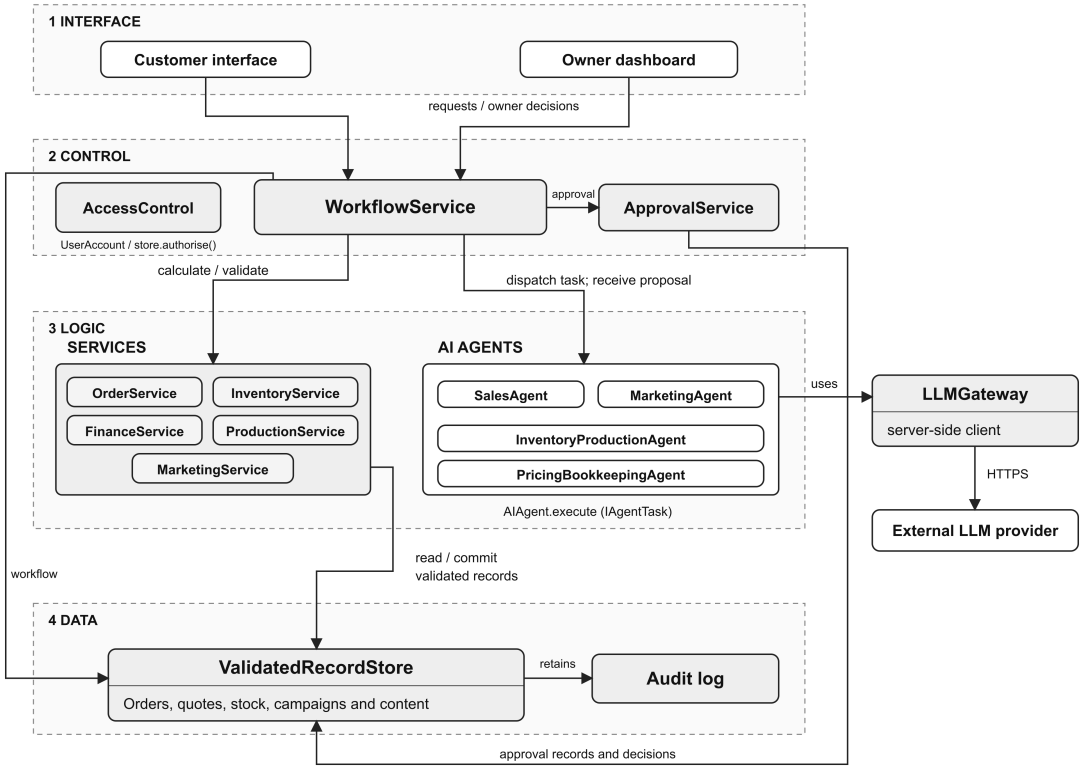


Figure 1: Ad hoc overall design showing conceptual layers, direct workflow coordination, shared records, owner approval and the server-side gateway to an external LLM provider.

4 Feature Diagram

Figure 2 presents the capabilities of BakeFlow AI across four business modules: Sales and Customer Service, Inventory and Production, Pricing and Bookkeeping, and Marketing. The hierarchy decomposes each module into its main features. Filled circles indicate mandatory features in the prototype; hollow circles indicate optional extensions.

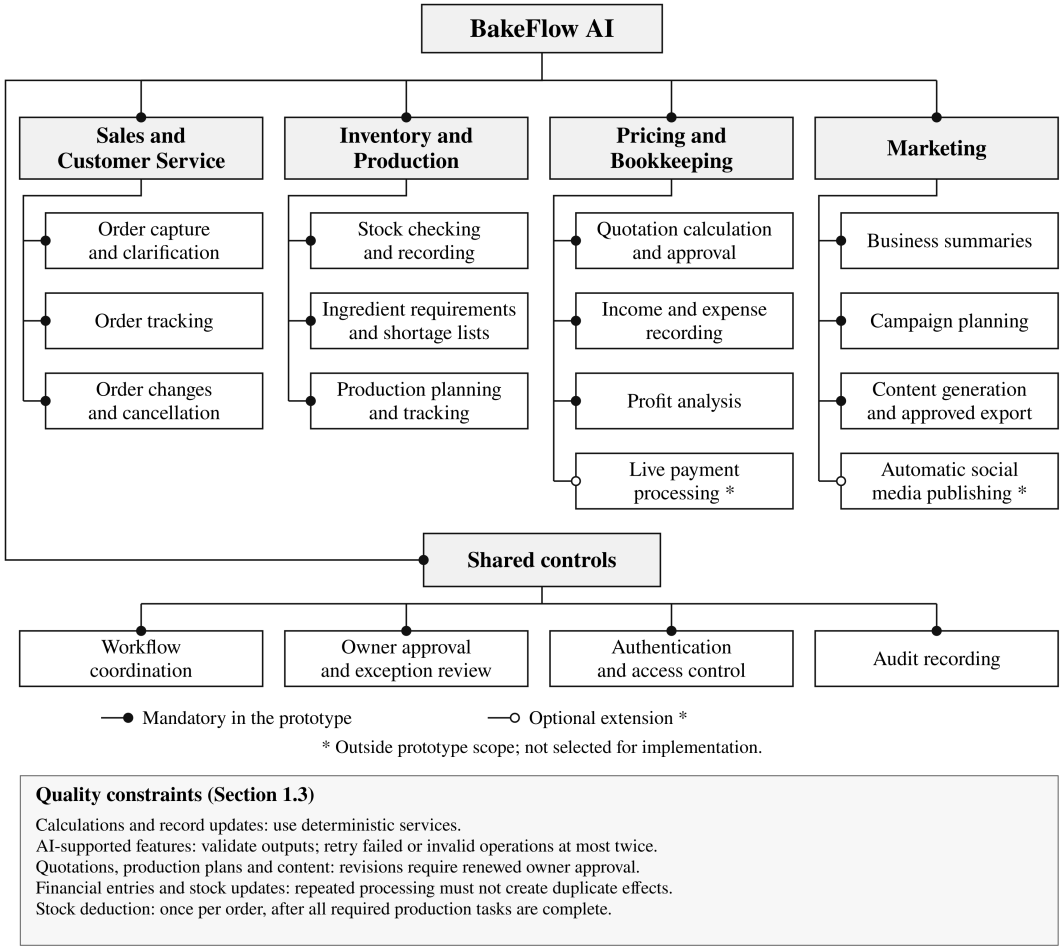


Figure 2: BakeFlow AI feature diagram showing mandatory capabilities, optional extensions and applicable quality constraints.

Shared controls are mandatory across the system: workflow coordination, owner approval and exception review, authentication and access control, and audit recording. These are shown as capabilities rather than implementation components; the later design models specify how they are realised.

Live payment processing and automatic social media publishing are optional extensions excluded from the prototype. Payments are recorded but not executed, and approved marketing content is exported for manual publication, consistent with Section 15.

The diagram also summarises the quality requirements and constraints specified in Section 1.3. Calculations and record updates use deterministic services, while AI

outputs require validation with at most two retries following failed or invalid operations. Revised quotations, production plans and marketing content require renewed owner approval. Repeated processing must not duplicate financial entries or stock deductions; ingredients are deducted once per order after all required production tasks are complete.

5 Use Case Overall Diagram

Figure 3 presents the eight principal use cases of BakeFlow AI, one owner extension use case and the three actors that interact with the system. The Customer and the Human Owner are the primary actors. The Customer submits orders, requests tracking, changes or cancellation, and accepts or rejects quotations. The Human Owner manages inventory, production, finances and marketing, approves quotations, and resolves order exceptions.

The LLM Provider is a secondary system actor: the external language-model service reached through `LLMGateway`, shown as a separate node in Figure 26. It supports all eight principal use cases, because each one invokes a specialist agent for interpretation, explanation, planning or content generation. It initiates no use case and has no authority over business records. The four AI agents and the Workflow Service operate within the system boundary and are therefore not shown as actors.

Two relationships are drawn directly from the specifications. *Create and Approve Production Schedule* «include»s *Check Inventory and Create Shopping List*, because step 2 of UC-04 performs the UC-03 stock check for every candidate order. UC-03 keeps its own association with the owner, who can also request a stock check on its own. *Review Order Exception* «extend»s *Submit Custom Order* and *Track, Change, or Cancel Order* at their *Owner Review* extension points, and applies only when the guard condition on each relationship holds. Its behaviour is specified by UC-01 flow 3.c and UC-02 flow 4.c rather than by a separate specification. The owner therefore takes part in UC-01 and UC-02 only through this extension, so no direct association links the owner to them. Ordering between use cases, such as quotation before scheduling, is controlled by the Workflow Service and shown in the activity diagrams rather than here.

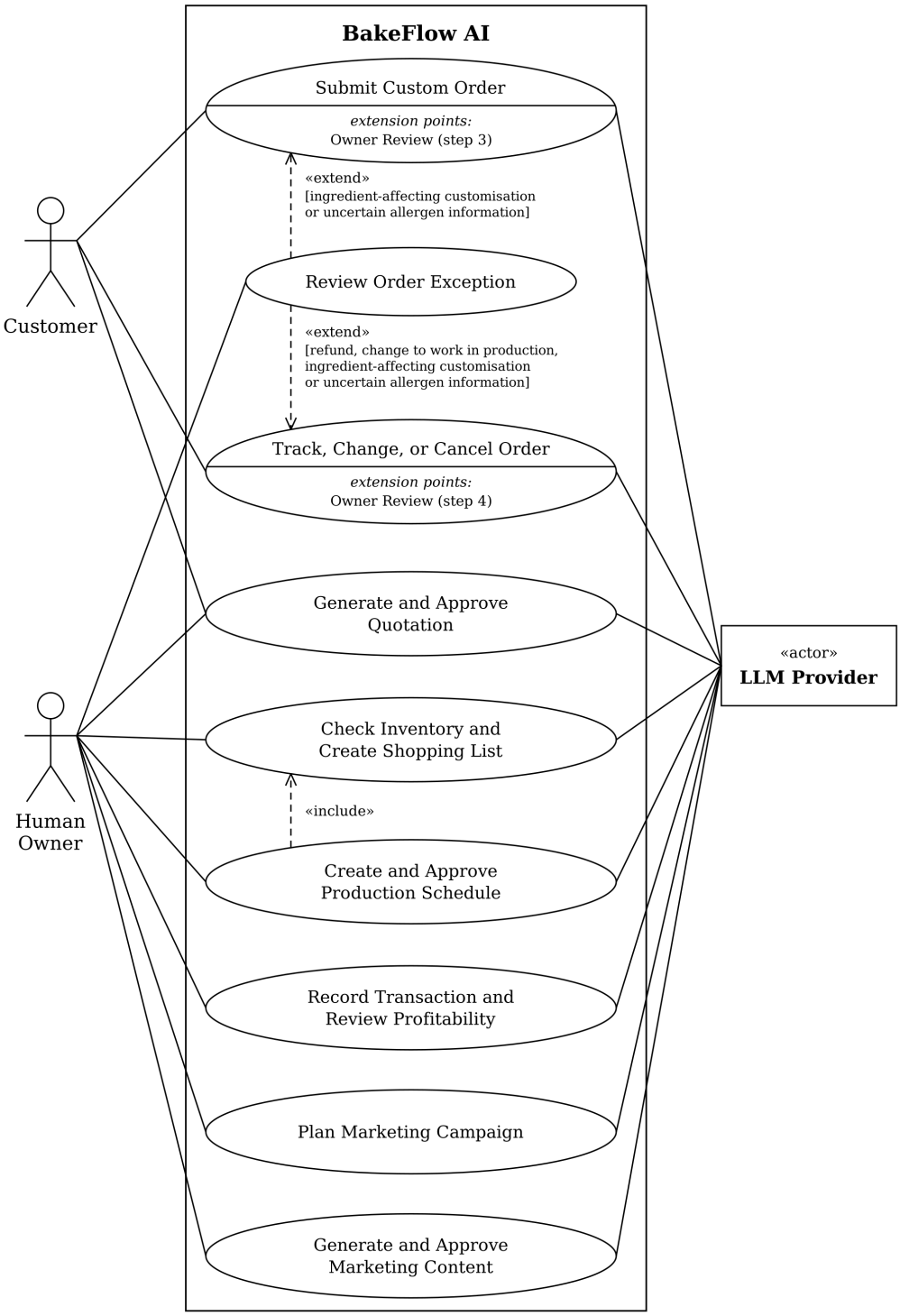


Figure 3: Overall use case diagram for BakeFlow AI, showing the Customer and Human Owner as primary actors, the LLM Provider as a secondary system actor, the eight principal use cases, the «include» of UC-03 by UC-04, and the owner exception review that «extend»s UC-01 and UC-02.

6 Use Case Specifications

6.1 Sales and Customer Service

6.1.1 UC-01: Submit Custom Order

Author: Ziyi Wang.

Goal: Capture the customer's requirements and save a validated draft order for inventory checking and quotation.

Level: User goal.

Primary actor: Customer.

Supporting actor: Human Owner, when manual review is required.

Preconditions: The customer is signed in and can access the ordering interface. Product and recipe information is available.

Trigger: The customer submits a custom order request.

Authority: The Sales Agent interprets and clarifies requirements. Deterministic services validate and save the order. The owner resolves ingredient-affecting customisations and uncertain allergen questions.

Main success scenario

1. The customer describes the products they want, including quantities, customisations and the required collection time.
2. The Sales Agent extracts the order details, including allergen concerns and any stated budget.
3. The system checks that the required information is complete, the values are valid and the requested products are supported.
4. The system presents an order summary to the customer.
5. The customer confirms that the summary accurately reflects their requirements.
6. The system performs final validation and saves the draft order against the customer, together with an audit record.
7. The system returns an order reference and informs the customer that inventory checking and quotation will follow.

Alternative and exception flows

- **3.a: Missing or unclear information.** The Sales Agent asks follow-up questions. The customer supplies or corrects the details, and the flow returns to Step 2. Budget is optional. Missing allergen information must not be interpreted as confirmation that there are no allergen concerns.

- **3.b: Unsupported product or customisation.** The system explains the limitation and asks the customer to revise the request. A revised request returns to Step 2. If no supported alternative is agreed, the submission ends without a validated draft.
- **3.c: Ingredient-affecting customisation or uncertain allergen information (extension point *Owner Review*).** The request is referred to the owner through *Review Order Exception* (Figure 3). The owner identifies a suitable product and recipe or requests further clarification. Once resolved, the details are rechecked at Step 3 and presented to the customer. The submission remains pending while the issue is unresolved.
- **5.a: Customer requests corrections.** The customer provides the changes, and the flow returns to Step 2.
- **During Steps 2–3: AI operation fails or produces invalid output.** The Workflow Service retries the AI operation at most twice after the initial attempt. If no usable result is obtained, the system informs the customer and stops the submission without saving an invalid draft.
- **6.a: Final validation fails.** The system identifies the details requiring correction and returns to Step 3.
- **6.b: Saving fails.** The system reports that the draft was not saved successfully and does not issue a successful submission confirmation. No partial business update is treated as successful.
- **Before Step 6: Customer withdraws the request.** The submission ends without creating a validated draft. Cancellation of an already saved order is handled by UC-02.

Success postcondition: A validated draft order is saved against the customer with an order reference and audit record. It is available for inventory checking and quotation. Inventory remains unchanged, and the sale is not yet confirmed; formal confirmation requires customer acceptance of an owner-approved quotation.

Failure postcondition: No new validated draft order is committed, and existing valid business records remain unchanged.

6.1.2 UC-02: Track, Change, or Cancel Order

Author: Ziyi Wang.

Goal: Allow the customer to view their order status and request a permitted change or cancellation.

Level: User goal.

Primary actor: Customer.

Supporting actor: Human Owner, when a request requires manual review or approval.

Preconditions: The customer is signed in and an order has been saved.

Trigger: The customer requests an order update, change or cancellation.

Authority: The Sales Agent interprets the request and explains recorded order information. The Workflow Service and deterministic services check permissions, apply order-state rules and save authorised changes. The owner handles refunds and changes affecting work already in production.

Main success scenario

1. The customer selects an order or provides its reference.
2. The system verifies that the customer is authorised to access the order and retrieves its current details and status.
3. The system presents the recorded order information. The Sales Agent explains the status and clarifies whether the customer wants to track, change or cancel the order.
4. The system handles the selected request. For tracking, it displays the current status and ends the use case without changing the order. For a change, it collects and clarifies the requested details, then checks whether the change is permitted in the current order state. For cancellation, it checks whether cancellation is permitted and presents the proposed outcome for customer confirmation.
5. For a change or cancellation, the customer reviews and confirms the request.
6. The system rechecks the current order state and saves the permitted change or cancellation together with an audit record. A change affecting a quotation or production plan is routed for the required checks and renewed approval before the affected work proceeds.
7. The system displays the saved result and clearly identifies any further review or approval still required.

Alternative and exception flows

- **2.a: Order unavailable or access denied.** The system does not disclose protected order information. It asks the customer to check the reference or sign in with the appropriate account. No order is changed.
- **4.a: Change request is incomplete or unclear.** The Sales Agent asks for clarification. The system repeats the checks once the customer supplies the required information.
- **4.b: Request is not permitted.** The system explains that the current order state does not allow the requested action. The order remains unchanged, and the customer may submit a different request.

- **4.c: Owner review is required (extension point *Owner Review*).** Refund requests, changes affecting work already in production, ingredient-affecting customisations and uncertain allergen questions are referred to the owner through *Review Order Exception* (Figure 3). The system records the request as pending without applying the requested business change. If the owner permits it, any revised terms are presented to the customer before the flow continues at Step 5. If the owner rejects it, the customer is informed and the requested change is not applied.
- **5.a: Customer withdraws the request.** The change or cancellation is not applied. The existing order remains unchanged.
- **6.a: Order state has changed.** The system does not apply a request based on outdated information. It retrieves the latest state and returns to Step 3 for review.
- **6.b: Saving fails.** The system reports that the change or cancellation was not saved successfully. It does not issue a success confirmation or treat a partial update as completed.
- **During interpretation or explanation: AI operation fails.** The Workflow Service retries the AI operation at most twice after the initial attempt. If an explanation remains unavailable, the system displays the recorded status with an explanation-unavailable message. If the customer's intended change cannot be reliably interpreted, no change is applied and the customer is asked to try again or seek owner assistance.

Success postcondition: The customer has received the current order status, or a permitted change or cancellation has been saved with an audit record. Changes affecting quotations or production plans require the relevant rechecking and approval. Cancelling an order does not by itself execute a refund or reverse recorded ingredient consumption.

Failure postcondition: No unauthorised or unsuccessful change is applied. The order remains unchanged by the failed request. Requests awaiting owner review remain pending and are not reported as completed.

6.2 Inventory and Production

6.2.1 UC-03: Check Inventory and Create Shopping List

Author: Chuanyang Wang.

Goal: Determine an order's ingredient requirements and show the owner what must be purchased.

Level: User goal.

Primary actor: Human Owner.

Preconditions: The owner is authenticated. The order has valid items and quantities. Ingredient-affecting customisations have been resolved to product recipes, with compatible ingredient units and current stock records available.

Trigger: The owner requests a stock check, or the Workflow Service identifies that an order needs an ingredient-availability check.

Authority: InventoryService calculates quantities and stock availability. The agent may explain the results but cannot alter them or purchase ingredients without the owner's action.

Main success scenario

1. The Workflow Service retrieves the order and its validated product/recipe information.
2. InventoryService calculates the ingredient quantities from order quantities, recipe yields and ingredient quantities per batch.
3. InventoryService checks current stock against these quantities, accounting for commitments to other approved orders under IP-02.
4. InventoryService creates a ShoppingList for this order. Its `shortages` map contains only positive shortfalls.
5. The system stores the validated result and its audit information through the existing record store.
6. The Inventory and Production Agent explains the shortages using the calculated quantities.
7. The system shows the required quantities, available stock and shopping list to the owner and returns the result to the Workflow Service.

Alternative and exception flows

- **1a–2a: Missing or invalid recipe data.** The system identifies an unresolved customisation, missing recipe, invalid quantity or incompatible unit and asks the owner to correct it. It does not publish an apparently complete shopping list from invalid data.
- **4a: No shortage.** The ShoppingList contains an empty `shortages` map and the system confirms that no additional ingredients are currently required.
- **5a: Save fails.** The system reports that the result was not saved and does not claim a successful persistent update. Stock quantities remain unchanged.
- **6a: Failed or invalid AI explanation.** The Workflow Service retries the AI operation at most twice. If it still fails, the owner receives the deterministic shopping list with an explanation-unavailable message; the calculated quantities remain valid.
- **7a: Ingredients are subsequently received.** The owner records actual received quantities. InventoryService validates and stores the stock increase, and the owner may repeat this use case. Generating a shopping list alone does not increase stock or place an order.

Success postcondition: A validated shortage-only ShoppingList is stored against the order and is available to the owner. The stock check itself has not deducted or increased inventory. An unavailable AI explanation does not invalidate the numeric result.

Failure postcondition: No new validated shopping list is claimed if the inputs or persistent save fail. Existing stock and previously valid records remain unchanged.

6.2.2 UC-04: Create and Approve Production Schedule

Author: Chuanyang Wang.

Goal: Produce a feasible, owner-approved production schedule for confirmed orders.

Level: User goal.

Primary actor: Human Owner.

Preconditions: The owner is authenticated. Orders are confirmed following acceptance of an owner-approved quotation. Their recipes, quantities and required times are known. Planning dates, daily production capacity and current stock records are available.

Trigger: The owner requests a new schedule or revises an existing production plan.

Authority: The Inventory and Production Agent proposes a plan. Deterministic services check stock and validate timing, capacity and task coverage. Only the owner approves the current schedule version.

Main success scenario

1. The Workflow Service retrieves the confirmed orders, planning constraints and relevant production records.
2. InventoryService calculates requirements and checks stock, using UC-03 for each candidate order and considering the combined commitments under IP-02. No candidate has an unresolved shortage.
3. The Workflow Service supplies the validated inputs to the Inventory and Production Agent through a TaskRequest.
4. The agent returns a proposed ProductionSchedule and its tasks. The Workflow Service checks the result's structure and confidence.
5. ProductionService validates that the tasks cover the candidate orders, fit the planning dates and daily capacity, and meet required times.
6. The system records the valid draft, requests approval for its current record reference and version, and presents it to the owner.
7. The owner approves the current version. ApprovalService records that decision and the deciding owner.
8. The system rechecks stock commitments, capacity and current-version approval. ProductionService performs a guarded save through the record store so a changed record cannot invalidate the checks unnoticed.
9. The system displays the saved, approved schedule and records the workflow transition and audit information.

Alternative and exception flows

- **2a: Insufficient ingredients.** The owner receives the shortage list. Planning pauses until stock is actually received or the order/plan is revised, then checking resumes.
- **4a: Failed, malformed or low-confidence AI proposal.** The Workflow Service retries at most twice. If no usable result is available, it reports the failure and leaves the plan unapproved.
- **5a: Infeasible schedule.** The system explains the capacity, timing or coverage issue. The owner changes the planning inputs and the system repeats proposal and validation.
- **7a: Owner requests revision.** The planned content is changed, the schedule version is incremented, and any active approval for the previous version becomes invalid. The system repeats checking and requests a fresh approval.
- **7b: Owner withholds approval.** The schedule remains unapproved and is not released for production. A later revision or decision resumes the process.
- **8a: Records changed during review.** A new stock commitment, capacity conflict or stale version prevents the save. The active approval cannot be used to release this plan; the owner is notified and the plan returns for checking and, where necessary, revision.
- **8b: Save fails.** The system reports the failure and does not display the plan as ready for production. No partial approved-plan update is treated as successful.

Success postcondition: A feasible schedule is saved with approval for its current version and an audit record. Production may proceed under that plan. Inventory has not been deducted by schedule creation or approval.

Failure postcondition: No new usable approved schedule is released. The proposal remains unapproved, requires revision, or awaits a successful save; stock remains unchanged by this use case.

Related execution constraint: Production execution is downstream of this approval use case. Task progress and once-only order deduction follow IP-01 and IP-04; approving a schedule is not equivalent to completing an order.

6.3 Pricing and Bookkeeping

6.3.1 UC-05: Generate and Approve Quotation

Author: Joshua Bates.

Goal: Produce an owner-approved quotation for a custom order and obtain the customer's decision.

Level: User goal.

Actors: Human Owner and Customer.

Preconditions: Order details and ingredient requirements are complete. Ingredient prices, labour costs, packaging costs and the owner's target profit margin are available. The owner is authenticated.

Trigger: The Workflow Service identifies that an order is ready for pricing.

Authority: The agent may explain the calculated quotation. Deterministic services calculate and store financial values. Only the owner may approve the final quotation.

Main success scenario

1. The system retrieves the order, ingredient requirements and relevant costing inputs.
2. `FinanceService` validates the inputs and calculates material, labour, packaging and additional costs.
3. `FinanceService` applies the owner-defined target profit margin to calculate the proposed price.
4. The Pricing and Bookkeeping Agent explains the quotation using the calculated cost breakdown, and the system checks that the explanation is consistent with those figures.
5. The system saves the draft quotation and presents its current version, cost breakdown and explanation to the owner.
6. The owner approves the quotation.
7. The system records the approval against the current quotation version and presents the approved quotation to the customer.
8. The customer accepts the quotation.
9. The system verifies that the accepted version is still current and owner-approved, records the acceptance and confirms the order through `OrderService`, recording the workflow transition.

Alternative and exception flows

- **2a: Invalid costing inputs.** The system identifies the invalid values and requests correction from the owner. No quotation is issued from invalid inputs, and calculation resumes after validation succeeds.
- **3a: Low-profit or unusually high quotation.** The system flags the quotation for explicit owner review before approval.
- **4a: Failed or invalid AI explanation.** The system retries the AI operation at most twice, then reports the failure to the owner and flags the quotation for manual review. The AI response does not change the calculated financial values.

- **6a: Owner requests changes.** The owner revises the quotation inputs. A new quotation version is created, approval of the previous version no longer applies, and the system repeats calculation, explanation and review before requesting renewed approval.
- **6b: Owner withholds approval.** The quotation remains unapproved and is not presented to the customer.
- **7a, 9a: Version or approval no longer current.** The system does not send the quotation (Step 7) or confirm the order (Step 9) on the basis of an outdated version. Fresh owner review is requested, and the customer is informed that a revised quotation will follow.
- **8a: Customer rejects the quotation.** The system records the rejection and informs the owner. If the owner proceeds with revised terms, Flow 6a applies, followed by renewed customer review.

Success postcondition: An owner-approved, customer-accepted quotation is stored against the order, the order is confirmed, and the approval and resulting workflow transition are recorded.

Failure postcondition: No accepted quotation is recorded and the order is not confirmed. The quotation remains unapproved, is rejected, or awaits revision, and the workflow does not proceed on the basis of customer acceptance.

6.3.2 UC-06: Record Transaction and Review Profitability

Author: Joshua Bates.

Goal: Record valid income or expenses and review updated order and product profitability.

Level: User goal.

Primary actor: Human Owner.

Preconditions: The owner is authenticated, and financial or order transaction data are available.

Trigger: An income or expense transaction is submitted.

Authority: Deterministic services validate transactions, maintain the ledger and calculate profit. The agent may interpret these results but may not independently alter financial records.

Main success scenario

1. The owner submits an income or expense transaction.
2. The system validates the transaction data and checks for duplicate entries.
3. `FinanceService` records the valid transaction in the ledger.

4. The system recalculates the affected order and product profitability using recorded income and expenses.
5. The Pricing and Bookkeeping Agent explains the updated results using the calculated figures.
6. The system displays the financial summary and explanation.
7. The owner reviews the updated profitability information.

Alternative and exception flows

- **2a: Invalid transaction.** The system rejects and logs the submission, identifies the invalid or missing data, and requests correction. Validation repeats if the owner resubmits the transaction.
- **2b: Duplicate transaction.** The system rejects and logs the duplicate and informs the owner. The existing ledger entry remains unchanged.
- **5a: Failed or invalid AI interpretation.** The system retries the AI operation at most twice, then reports the failure to the owner. The valid transaction and deterministic profit calculations remain available for review.

Success postcondition: The valid transaction is stored once, profitability figures are updated, and the owner can review the financial summary.

Failure postcondition: An invalid or duplicate transaction causes no change to the ledger or profitability figures. If only the AI interpretation fails, the valid financial records and calculated results are retained.

6.4 Marketing

6.4.1 UC-07: Plan Marketing Campaign

Author: Angelina.

Goal: Produce an owner-approved marketing campaign plan based on validated business data.

Level: User goal.

Primary actor: Human Owner.

Preconditions: Sales, inventory, profit and production-capacity summaries are available. The owner is authenticated.

Trigger: The owner requests a new campaign.

Authority: Deterministic services calculate and supply the business summaries. The Marketing Agent may interpret those summaries and propose a campaign, but only the owner may approve a campaign plan.

Main success scenario

1. The Workflow Service validates the request and routes the task to the Marketing Agent.
2. MarketingService retrieves validated sales, profit, stock and production-capacity data and assembles a business summary.
3. The Marketing Agent reads the validated business summary.
4. The Marketing Agent identifies products suitable for promotion.
5. The Marketing Agent evaluates seasonal events, stock levels, profitability and available production capacity.
6. The Marketing Agent recommends campaign objectives, products and timing.
7. The Workflow Service submits the proposed campaign plan for owner approval.
8. The owner reviews and approves the campaign plan.
9. The system stores the approved campaign and records the approval decision with an audit entry.

Alternative and exception flows

- **2a: Insufficient business data.** The Marketing Agent may produce a general campaign suggestion with a low-confidence notice. A structurally valid suggestion that passes applicable business checks may proceed to explicit owner review. The notice does not waive validation or approval; unusable or invalid output follows flow 6a.
- **6a: Failed or invalid AI output.** The Workflow Service retries the AI operation at most twice, then reports the failure to the owner. No campaign record is created from invalid output.
- **8a: Owner requests revision.** The Marketing Agent revises the campaign plan, the version is incremented, and any approval of the prior version no longer applies. The revised plan is checked and resubmitted for approval.
- **8b: Owner rejects the plan.** The rejection is recorded and no campaign proceeds.

Success postcondition: An owner-approved campaign plan is stored against the current version, with the approval decision recorded in the audit log.

Failure postcondition: No approved campaign is stored. The plan remains unapproved, is rejected, or awaits revision, and no dependent marketing work proceeds.

6.4.2 UC-08: Generate and Approve Marketing Content

Author: Angelina.

Goal: Produce owner-approved promotional content for an approved campaign.

Level: User goal.

Primary actor: Human Owner.

Preconditions: An approved campaign plan exists. The owner is authenticated.

Trigger: The owner requests promotional content for the campaign.

Authority: The Marketing Agent may draft content. Deterministic services validate referenced products, prices and availability. Only the owner may approve public content, and publication remains manual.

Main success scenario

1. The Workflow Service routes the content task to the Marketing Agent.
2. The Marketing Agent reads the approved campaign plan.
3. The Marketing Agent generates social media content and product descriptions.
4. MarketingService validates the referenced products, prices and availability against stored records.
5. On entering AwaitingOwnerDecision, the Workflow Service creates one PublicContent approval request for the current content version and presents the validated draft to the owner.
6. The owner reviews the generated content.
7. The owner approves the current content version.
8. ApprovalService records the decision for that version. The system checks that the version and approval are current and that isReadyForExport() succeeds before adding the content to the calendar for manual export.

Alternative and exception flows

- **3a: Failed or invalid AI output.** The Workflow Service retries the AI operation at most twice, then reports the failure to the owner. No content record is stored from invalid output.
- **4a: Validation failure.** Content referencing unavailable products or incorrect prices is returned to draft with the validation errors reported, and is regenerated before it can be reviewed.

- **7a: Owner requests another version.** The owner requests a different tone, platform format or content version. The content returns to draft, the version is incremented, and any earlier approval cannot authorise the new version. It is validated and reviewed again.
- **7b: Owner rejects the content.** The rejection is recorded. The content may be revised and resubmitted, or abandoned.
- **8a: Approved content is subsequently edited.** This applies both before and after the content is added to the calendar. Editing increments the version, invalidates the existing approval and returns the content to Draft. The revised version cannot be exported until it has been validated and approved again.

Success postcondition: Owner-approved content is stored against the approved version, scheduled in the content calendar and available for manual publication, with the approval recorded in the audit log.

Failure postcondition: No failed, rejected or unapproved content version becomes eligible for export. The current proposal remains in Draft, is rejected, or awaits revision; earlier records and audit decisions are retained without authorising an edited version.

7 Architecture Analysis and Design

7.1 Architectural drivers

The architecture of BakeFlow AI is shaped by four properties of the problem. The business process is discrete and state-based: an order moves through a small number of identifiable states and must not skip a required step. Exact correctness is required wherever a language model cannot be trusted, because prices, stock quantities and ledger entries must be reproducible and auditable. Language-model calls are expensive and non-deterministic, so they must be confined to the tasks that genuinely need them. Several decisions carry commercial or safety consequences — pricing, refunds, allergen exceptions and public content — and therefore require an explicit human decision. The patterns below are selected to satisfy these drivers rather than for generality.

7.2 Selected architectural patterns

Layered architecture. The interface, control, logic and data layers separate user interaction, workflow coordination, business behaviour and persistence. Calls may cross these conceptual boundaries. `WorkflowService` directly invokes agents and services, and the workflow, business and approval services may access `ValidatedRecordStore`. Validation and access checks protect persistent updates; AI agents do not write business records directly.

Workflow-controlled architecture. `WorkflowService` applies predefined states and transition rules. An order proceeds through inventory checking, quotation generation, owner approval, customer acceptance and production scheduling in the required order. Marketing follows campaign planning and approval before content generation and approval. Each use case defines its checks, failure handling and approval gates; agents do not decide the overall routing.

Shared-data architecture. Components use the common `ValidatedRecordStore` for structured records such as Order, Quote, Transaction, InventoryItem, Campaign and MarketingContent. It retains validated drafts, approved versions, approval decisions and audit information. Shared persistence provides a traceable source of business data alongside direct service calls. It is not a blackboard mechanism in which independent agents schedule themselves solely by observing shared records.

Event-triggered workflow progression. Business events such as order submission, quotation approval and production completion trigger predefined transitions in `WorkflowService`. The design uses direct calls within one application runtime; it does not require an event bus or publish–subscribe infrastructure. New reactions require explicit workflow rules and any related service changes, so event-triggered behaviour does not imply independent or loosely coupled modules.

7.3 Rationale

Together these choices keep exact calculation in deterministic services and AI output subject to validation. Failed or unusable AI results follow the retry and escalation rules

Pattern	Advantages	Costs and limitations
Layered	Separates interfaces, coordination, AI proposals, deterministic services and persistence. Clear responsibilities support focused validation and testing.	Layers still have explicit service and data dependencies. Shared record or interface changes may require coordinated updates across layers.
Workflow-controlled	Makes routing, required checks and approval gates explicit and auditable. Transition rules can be tested independently of model output.	The workflow service is a central point of change. New steps and exception paths require rule changes and corresponding tests.
Shared-data	Provides consistent structured records, versioned approvals and traceable updates without relying on informal agent-to-agent messages.	The store is a shared dependency and potential point of contention or failure. Schema, version and access rules must be coordinated across modules.
Event-triggered	Matches the discrete business lifecycle and makes the triggers for progress, pauses and renewed review explicit.	Direct calls and shared schemas remain coupled. New reactions need workflow updates; retries and repeated events require guarded transitions and audit records.

Table 1: Advantages and limitations of the architectural choices.

of the relevant use case, while independently valid service results remain available. Shared records and the audit log support traceability, and current-version approval gates preserve human control. Language-model calls are limited to interpretation, explanation, planning and content generation.

7.4 Selection of viewpoints

Figure 4 summarises the 4+1 architecture views.

The **logical view** identifies `WorkflowService`, the four agents, services, records and approvals, refined by the class model in Section 8. `IAgentTask` is the informal label for the `AIAgent.execute` contract; `LLMGateway` implements `ILLMClient`. The **development view** groups four business modules with coordination, AI support and shared model/services, as detailed by the package diagram in Section 13. These groups retain explicit service and shared-data dependencies.

The **process view** shows predefined workflow transitions, direct calls to agents and services, validation, current-version owner decisions and audited commits. The activity, interaction and state models in Sections 10–12 refine this behaviour. The **physical view** places browser interfaces, one application host, a local SQL database and the server-side LLM gateway alongside an external LLM provider, as detailed in Section 14. The **scenarios** connect the views through UC-01, UC-05, UC-04 and UC-08.

The process view is primary because it shows business sequencing, owner decisions, validation failures and revision. The other views describe the supporting structure and deployment.

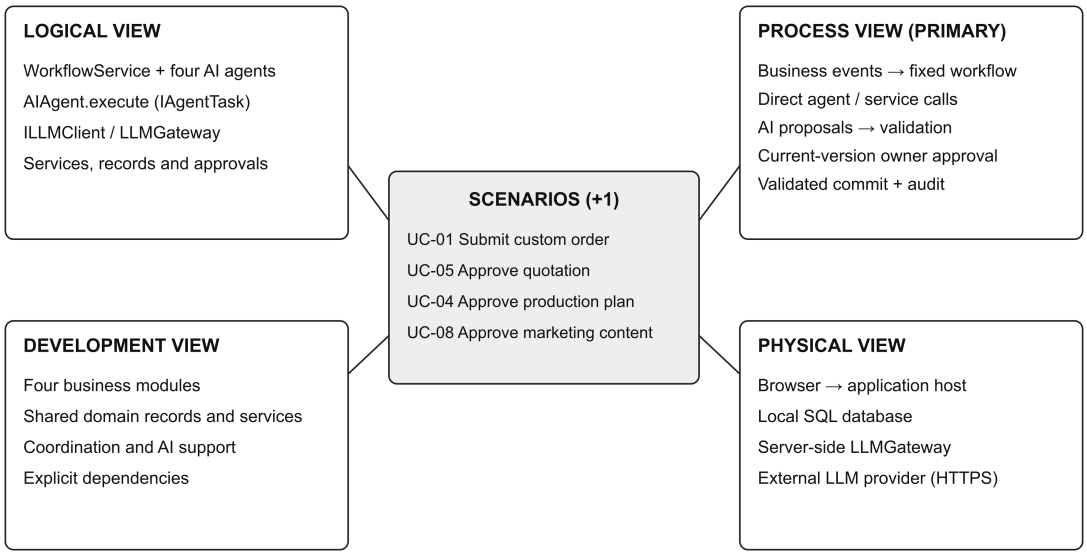


Figure 4: The 4+1 architecture views, with the process view as the primary viewpoint and representative use cases linking the views.

8 Elementary Structure Modelling

The BakeFlow AI class model describes the system's design-time structure through four views: agents and workflow, orders and finance, inventory and production, and marketing and approvals. The views form one model; repeated class names refer to the same class. Each class shows its principal attributes and operations. Relationships use UML associations, generalisation, composition, aggregation and interface realisation, with multiplicities indicating the permitted numbers of related objects.

Figure 5 shows the deterministic `WorkflowService` and four specialised AI agents. The agents inherit common attributes and operations from the abstract `AIAgent` class and provide role-specific behaviour. `IAgentTask` in Figures 1 and 4 is an informal label for the common task contract that every agent implements; in the class model this contract is the abstract operation `execute(t: TaskRequest): TaskResult` on `AIAgent`, rather than a separate interface. Each agent uses one `ILLMClient`, implemented by `LLMGateway`, allowing the LLM integration to change without changing the agent classes. The workflow service coordinates agent tasks and access to deterministic services. AI outputs remain proposals that require validation before persistent business records are changed.

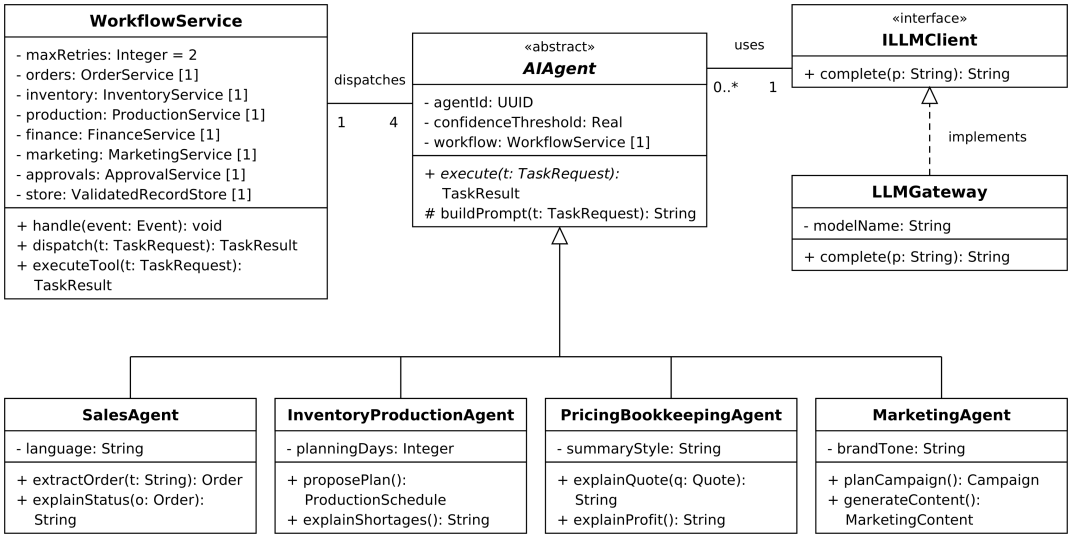


Figure 5: Class diagram of workflow coordination, specialised AI agents and the LLM interface.

Figure 6 models order capture, quotation generation and financial recording. A customer can place multiple orders, while each order belongs to one customer. An order contains one or more `OrderItem` objects through composition; each item references one product. An order can have multiple quotations, allowing revised proposals to be retained. The `FinanceService` performs deterministic quotation and profitability calculations. Quotes require owner approval before customer acceptance, and an accepted quote is required before order confirmation. Transactions may reference an order and product, while general expenses can exist without either association.

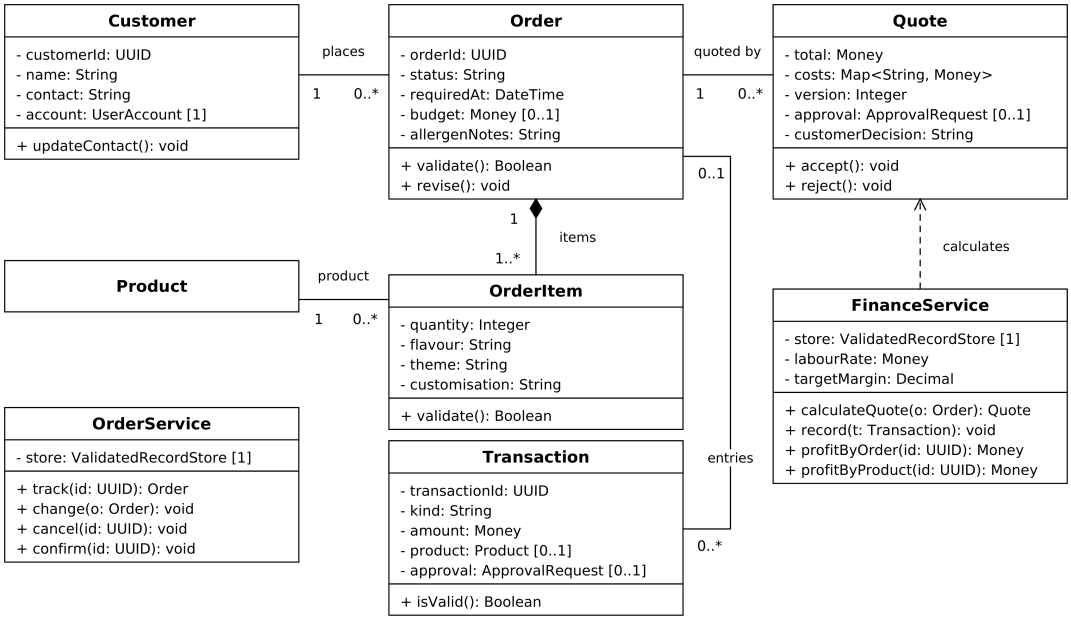


Figure 6: Class diagram of customers, orders, quotations and financial services.

Figure 7 connects products and recipes to ingredient stock and production planning. The design assumes one recipe per product. Each recipe owns its ingredient entries through composition, while the referenced inventory items exist independently. The `InventoryService` calculates ingredient requirements, checks availability and creates a shopping list of shortages. The `ProductionService` validates proposed schedules against capacity and timing constraints. A schedule owns its production tasks, and each task references one order. Production proceeds only after schedule approval, with stock deducted once per order following completion.

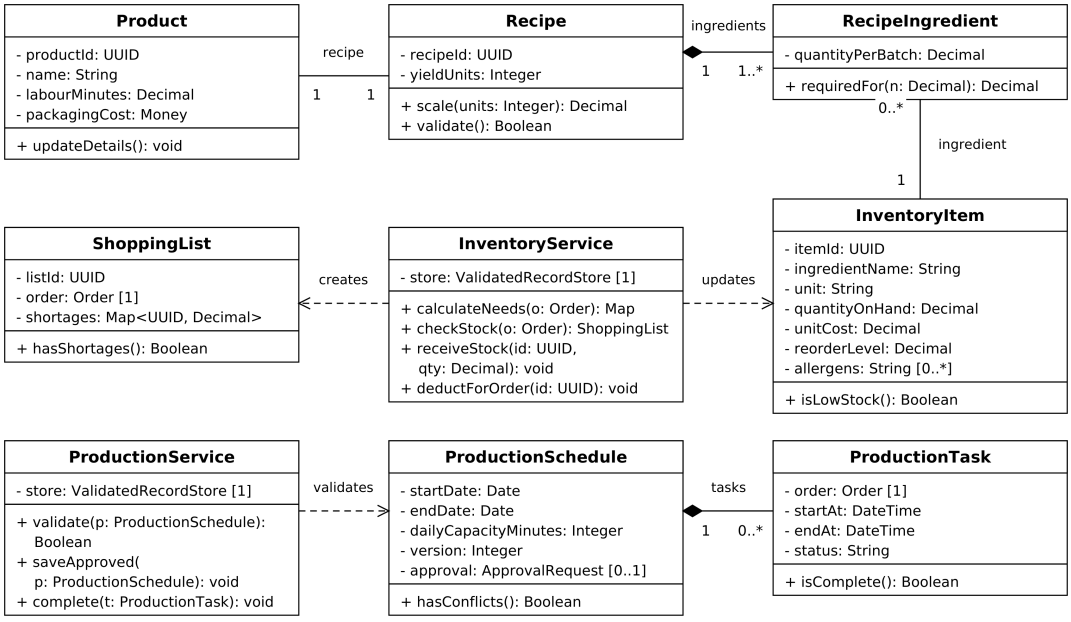


Figure 7: Class diagram of recipes, inventory, shopping lists and production schedules.

Figure 8 models marketing records and shared approval and persistence functions. A campaign groups existing products through aggregation, allowing products to remain independent of campaign lifecycles. Marketing content belongs to its campaign through composition. The `ApprovalService` records decisions against a particular record version; subsequent edits invalidate the previous approval. An approval request may have no deciding user while pending, but only an owner account may approve it. The `ValidatedRecordStore` authorises access and commits validated changes with audit entries recording the responsible actor, timestamp and state change.

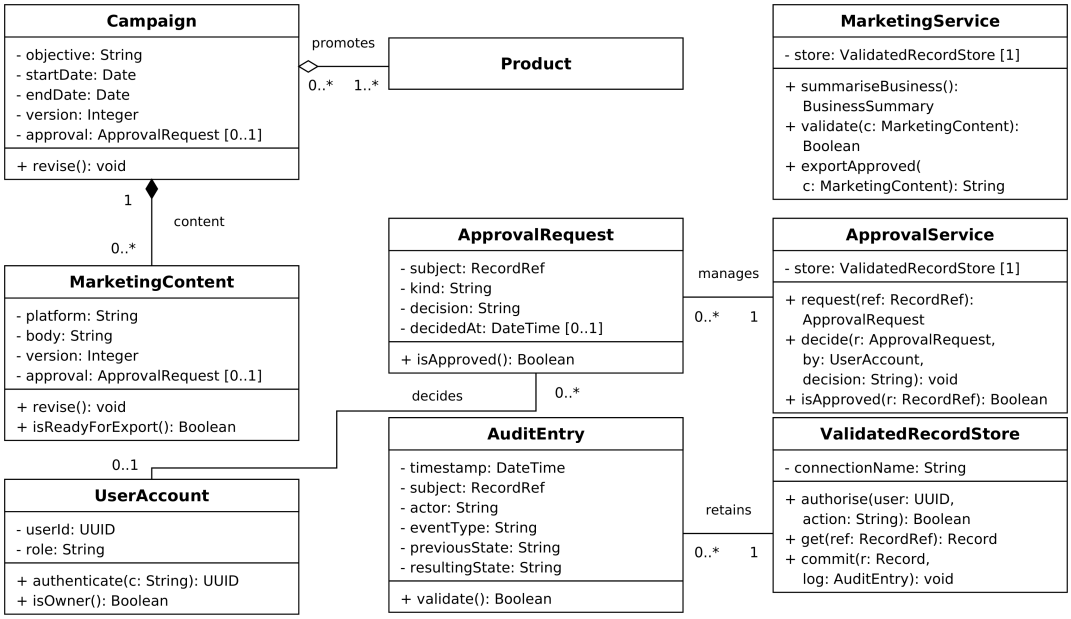


Figure 8: Class diagram of campaigns, marketing content, owner approvals and shared record storage.

Supporting types represent structured data exchanged between classes. `TaskRequest` contains a task identifier, task kind and input data; `TaskResult` contains a status, confidence value and proposed record. `Event` identifies an event, its subject and actor. `RecordRef` contains a record type, identifier and version, while `Record` holds the corresponding business data. `BusinessSummary` contains sales, profit, stock and capacity summaries. Ingredient quantity maps associate inventory item identifiers with decimal quantities, and monetary amounts use two decimal places.

9 Complex Structure Modelling

The runtime model instantiates the existing class model at one illustrative moment: 1 October 2026 at 08:35, after a production schedule has been approved but before production starts. Two views show the same snapshot rather than two successive workflow stages. Object names are underlined and each box shows selected attribute values; solid links represent existing associations or reference-valued properties, not method calls.

Figure 9 shows all four concrete agents sharing one LLMGateway and referring to the deterministic WorkflowService. The workflow coordinates the selected inventory, production and approval services, which share the ValidatedRecordStore. The OrderService, FinanceService and MarketingService references are omitted from this view for readability, not removed from the design. No interface or abstract agent is instantiated.

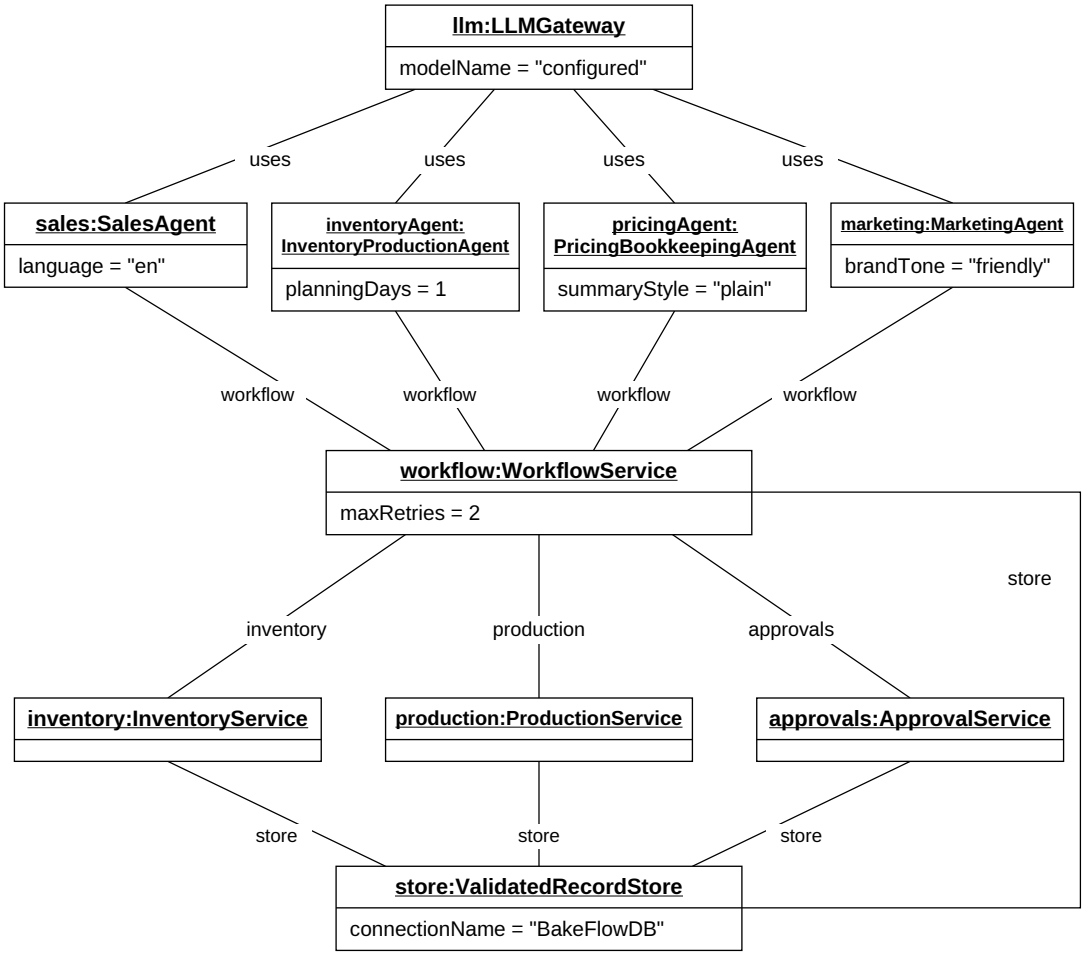


Figure 9: Runtime service collaboration: concrete agent and service objects in the approved-schedule snapshot. Only selected references are expanded.

The links show the collaboration structure available at this time. They do not imply that all agents are executing simultaneously. The Inventory and Production Agent proposes plans; the deterministic services validate and persist business changes through their existing interfaces.

Figure 10 expands a confirmed order for 24 vanilla cupcakes. Its accepted quotation and approved schedule lead to two planned tasks, `bake` and `finish`, both referencing the same order. The owner's approval identifies schedule version 2. The RecordRef tuple contains the record type, identifier and version. Short IDs such as `U1`, `O1` and `S1` are aliases for illustrative identifiers, not new class attributes.

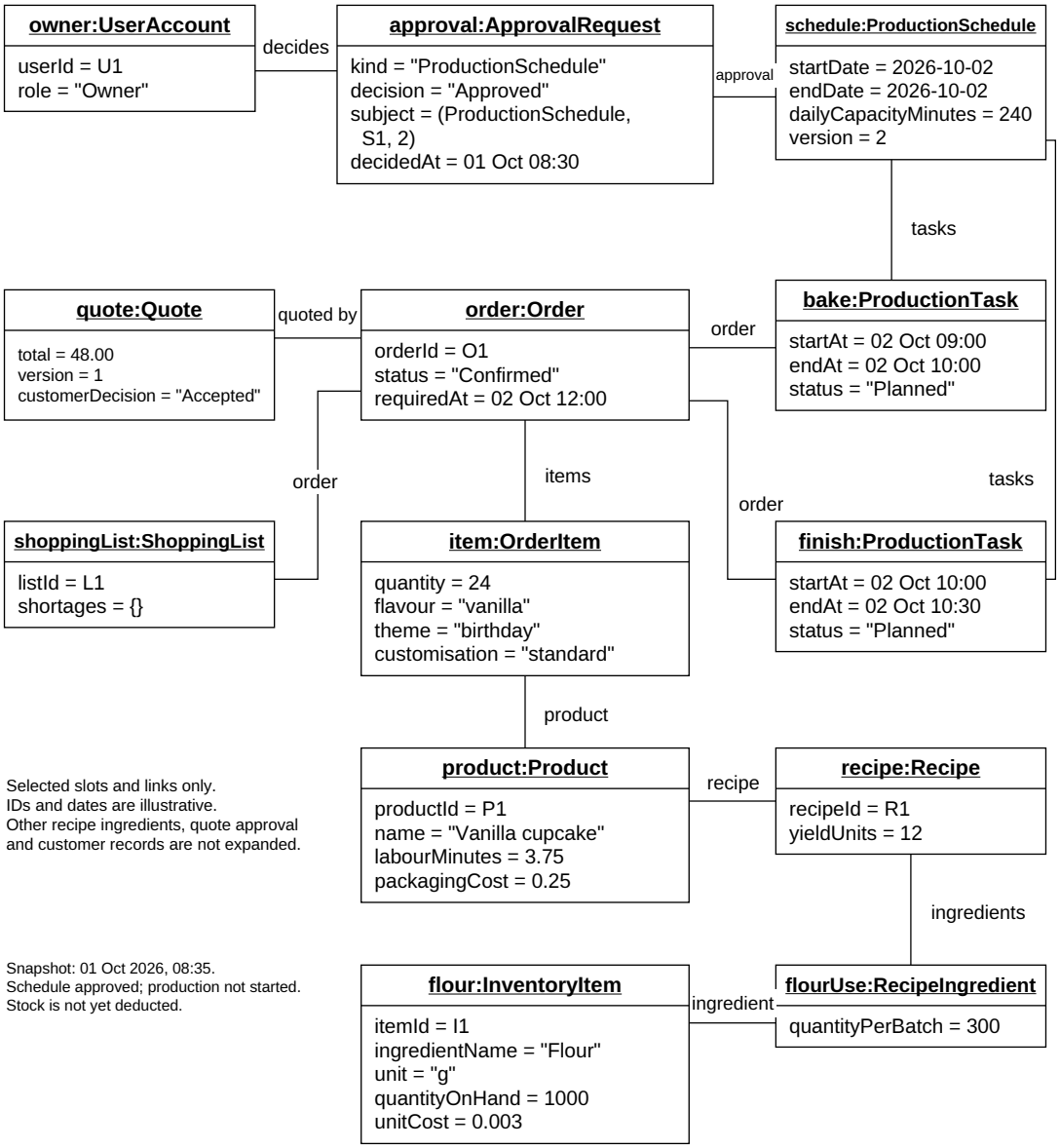


Figure 10: Business-object view of the same snapshot: confirmed order, accepted quotation, approved schedule, planned tasks and recipe-based inventory data.

The displayed recipe yields 12 cupcakes and uses 300 g of flour per batch, so the order requires $24/12 \times 300 = 600$ g. No other flour commitment is assumed, and omitted ingredients are sufficient; the shortage map is empty. Stock remains 1000 g before production. Completing both tasks would cause one 600 g deduction, not two. Customer records, quotation approval and other ingredients are omitted from this partial view. All values are illustrative, not observed bakery records.

10 Activity Diagrams

10.1 Sales and Customer Service

Figure 11 models UC-01, *Submit Custom Order*, and supports requirement AR-01. The Sales Agent extracts the customer's requirements, and the system checks whether the details are complete and valid. Missing or unclear information is returned to the customer for clarification. Ingredient-affecting customisations and uncertain allergen questions are referred to the owner. The customer reviews the order summary and may request corrections before the system validates and saves the draft.

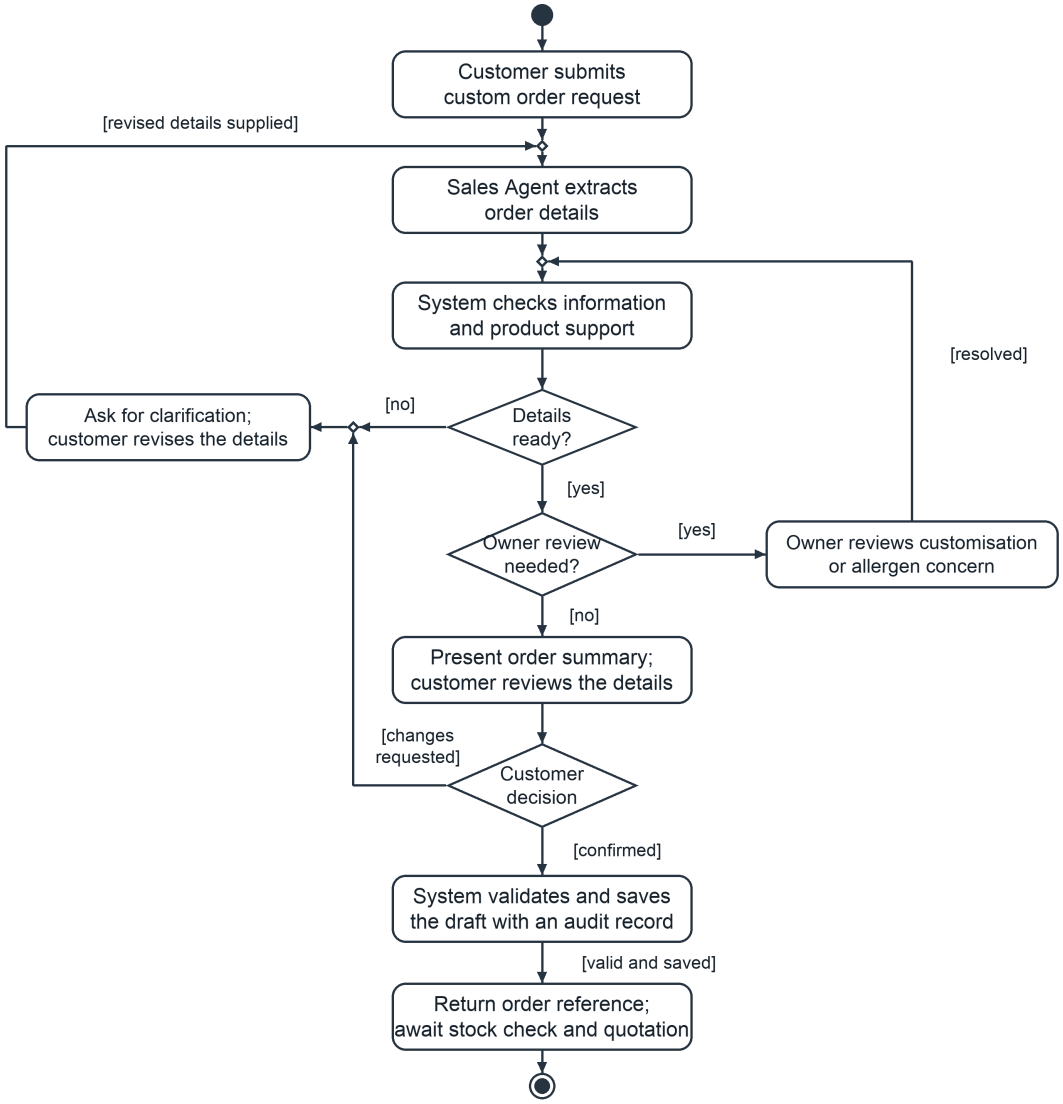


Figure 11: Activity diagram for custom order submission, clarification, owner review and draft saving.

The diagram focuses on the main flow and key alternative paths. AI retries, customer withdrawal and failure handling are described in UC-01 rather than expanded here. Unresolved owner review prevents further progress, and an order reference is returned only after the draft is successfully saved. The final node represents completion of

draft submission, not confirmation of a sale. Inventory checking and quotation follow separately, and no stock is deducted in this activity.

10.2 Inventory and Production

Figure 12 models UC-04 and AR-02, using UC-03 for stock checking. Rounded nodes are actions; diamonds are decisions or merges. Shortages and infeasible plans return for correction. The Workflow Service retries unusable AI output at most twice, then pauses if necessary. Owner approval is followed by fresh checks and a guarded save.

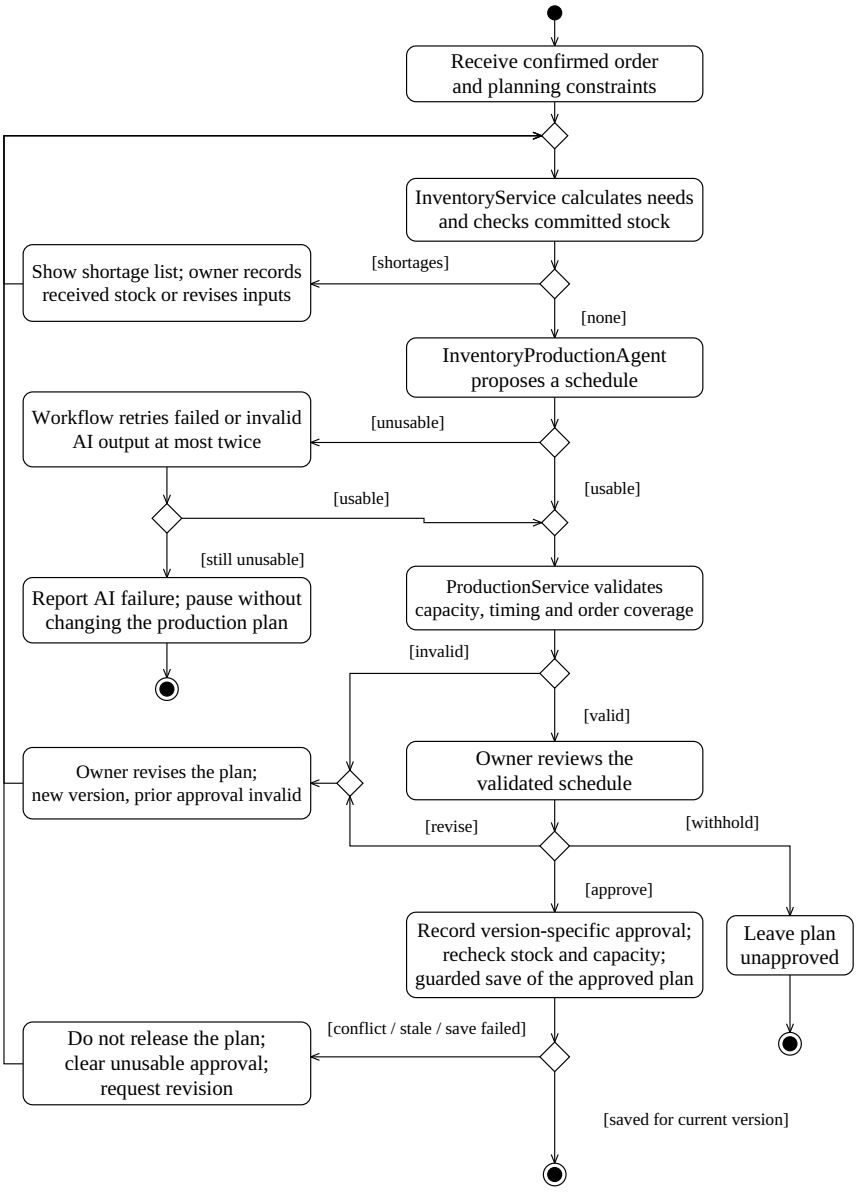


Figure 12: UC-04 activity diagram: stock checking, AI proposal, deterministic schedule validation, owner review and guarded approval.

The final nodes end this planning attempt, not the order's lifetime. No inventory is

deducted here. Planned content changes require a new version; task-progress updates are outside this activity and follow IP-04.

10.3 Pricing and Bookkeeping

Figure 13 models UC-05, *Generate and Approve Quotation*, and realises requirement AR-03. Partitions show which participant performs each action. `FinanceService` validates the costing inputs, returning invalid values to the owner for correction (flow 2a), then calculates material, labour, packaging and additional costs and applies the owner-defined target profit margin to produce a draft `Quote`. A fork then runs two independent branches: the `PricingBookkeepingAgent` explains the quotation while `FinanceService` checks for low profit or an unusually high price. `WorkflowService` checks the explanation against the calculated breakdown and retries a failed or invalid explanation at most twice before reporting the failure to the owner. After the join, a quotation with an exceptional price or no usable explanation is flagged for manual review (flows 3a and 4a); neither condition alters the calculated values. The owner may approve the current version, withhold approval or revise the inputs, and a revision creates a new quote version that invalidates the previous approval (flow 6a). Only the approved version is sent to the customer. On acceptance, `WorkflowService` confirms that the version is still current and approved before `OrderService` confirms the order (flow 9a); on rejection, the owner may revise the terms and re-enter calculation (flow 8a).

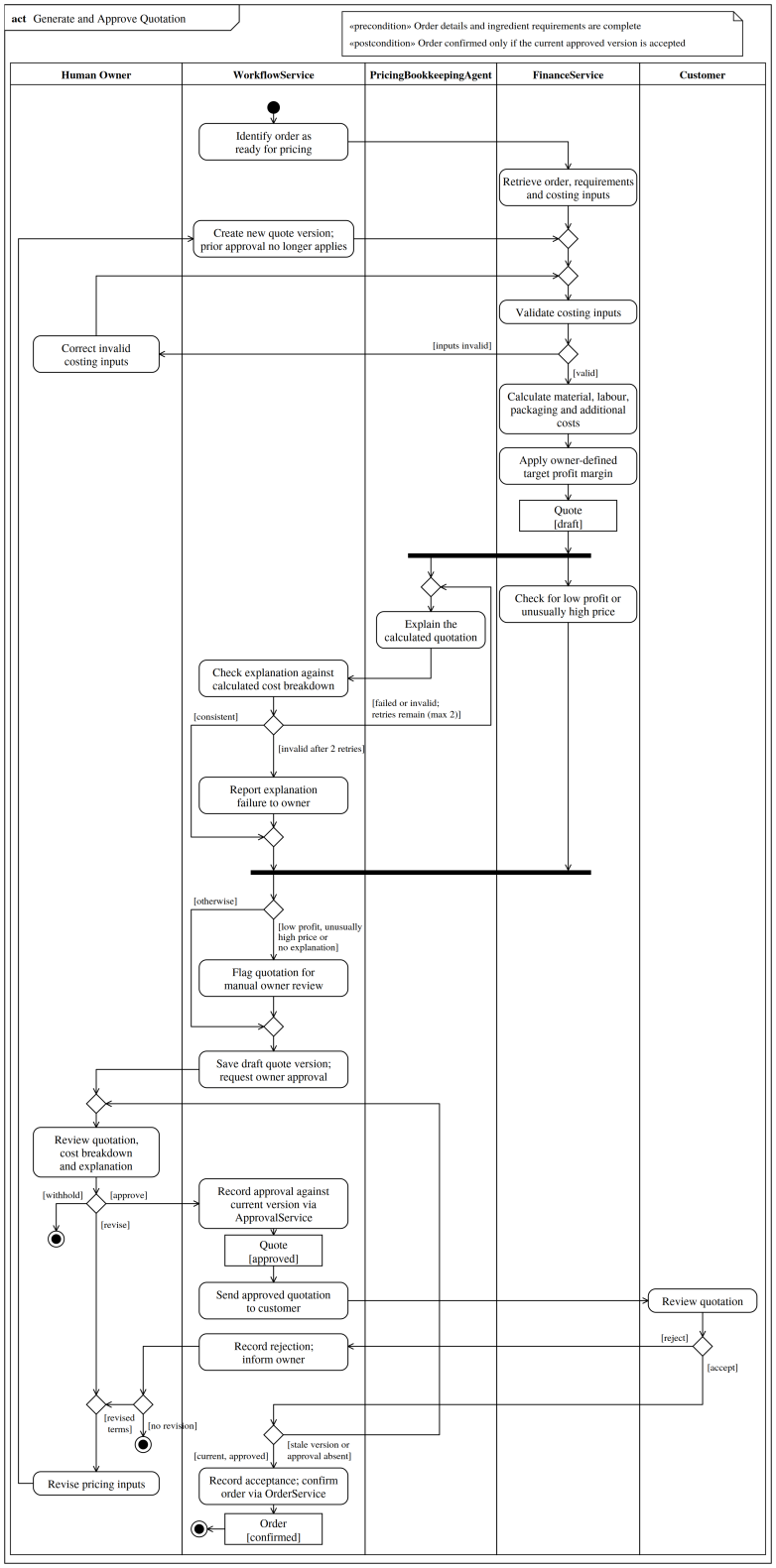


Figure 13: UC-05 activity diagram with partitions, showing input validation, concurrent AI explanation and price checking, owner approval or revision, customer decision and order confirmation.

10.4 Marketing

Figure 14 models the campaign planning workflow for AR-04 and UC-07, *Plan Marketing Campaign*. The partitions show where each action is performed and make the separation between the owner, the rule-based control layer, the AI agent and the deterministic services explicit.

After the owner requests a campaign, the Workflow Service validates the request and routes the task. MarketingService retrieves sales and profit data in parallel with stock and production-capacity data, and the fork and join ensure that the business summary is assembled only once both have been returned. The Marketing Agent then evaluates whether the summary contains sufficient data. If it does, the agent identifies products suitable for promotion and recommends campaign objectives, products and timing. If it does not, the agent produces a general campaign suggestion carrying a low-confidence notice, which is the alternative flow of UC-07. Both paths merge for owner review only when the proposal is structurally valid and passes applicable business checks. Limited data may justify a low-confidence notice; it does not justify invalid content. Failed or unusable AI output follows the retry and escalation rules in UC-07.

The plan is then submitted for owner approval. An approved plan is stored as a Campaign record with its approval, a rejected plan is recorded and stops, and a request for revision returns the plan to the Marketing Agent and re-enters the approval step. No campaign proceeds without an explicit owner decision.

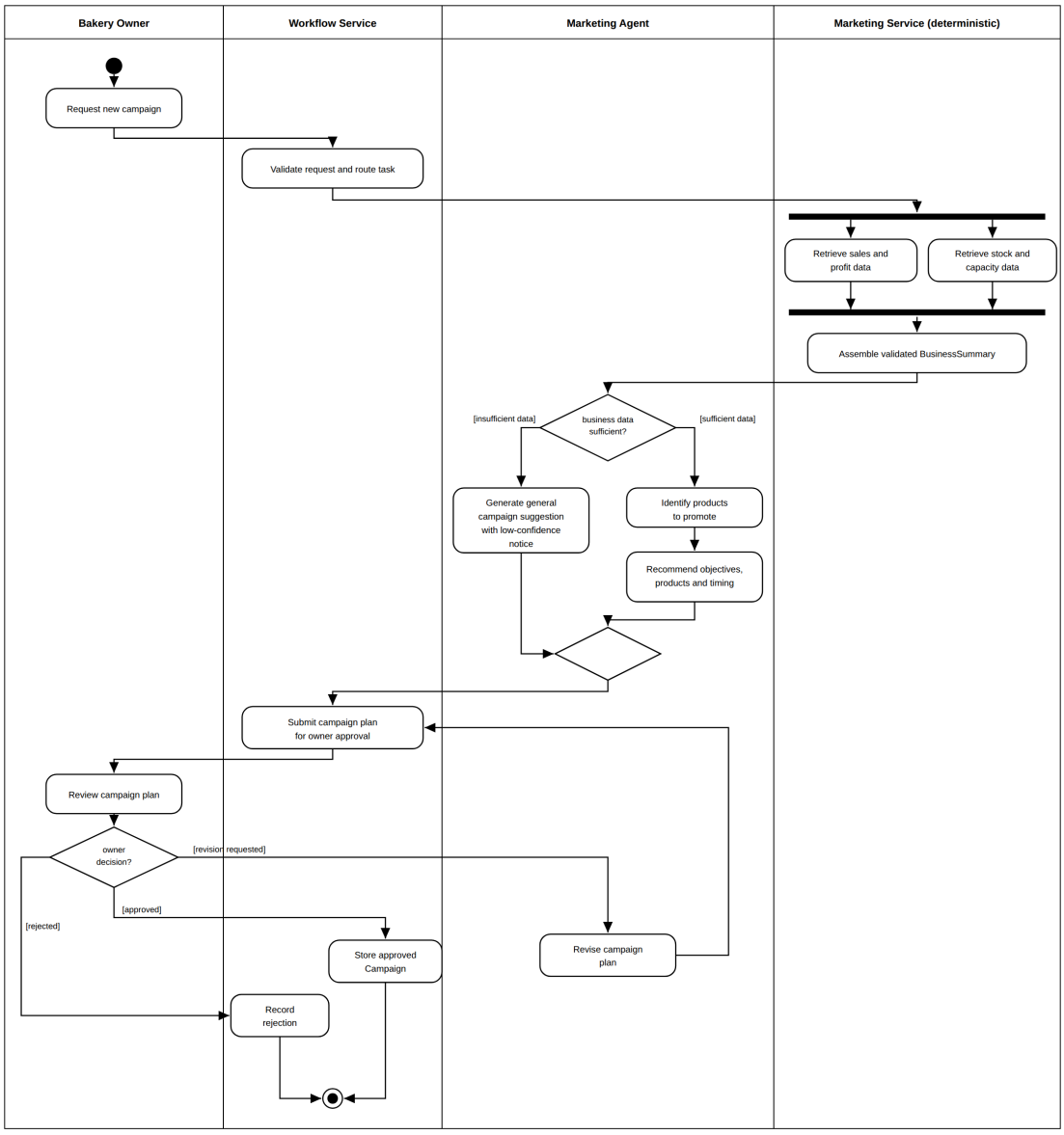


Figure 14: Activity diagram for marketing campaign planning, showing parallel retrieval of business data, the low-confidence alternative flow, and owner approval, rejection or revision.

11 Interaction Diagrams

11.1 Sales and Customer Service

Figure 15 models the main success path and clarification loop of UC-01, *Submit Custom Order*, supporting AR-01. The Workflow Service sends the customer's request to the Sales Agent and checks the proposed order details. Missing or invalid information triggers clarification and repeated validation. The customer confirms the summary before the draft is validated again and saved with an audit record.

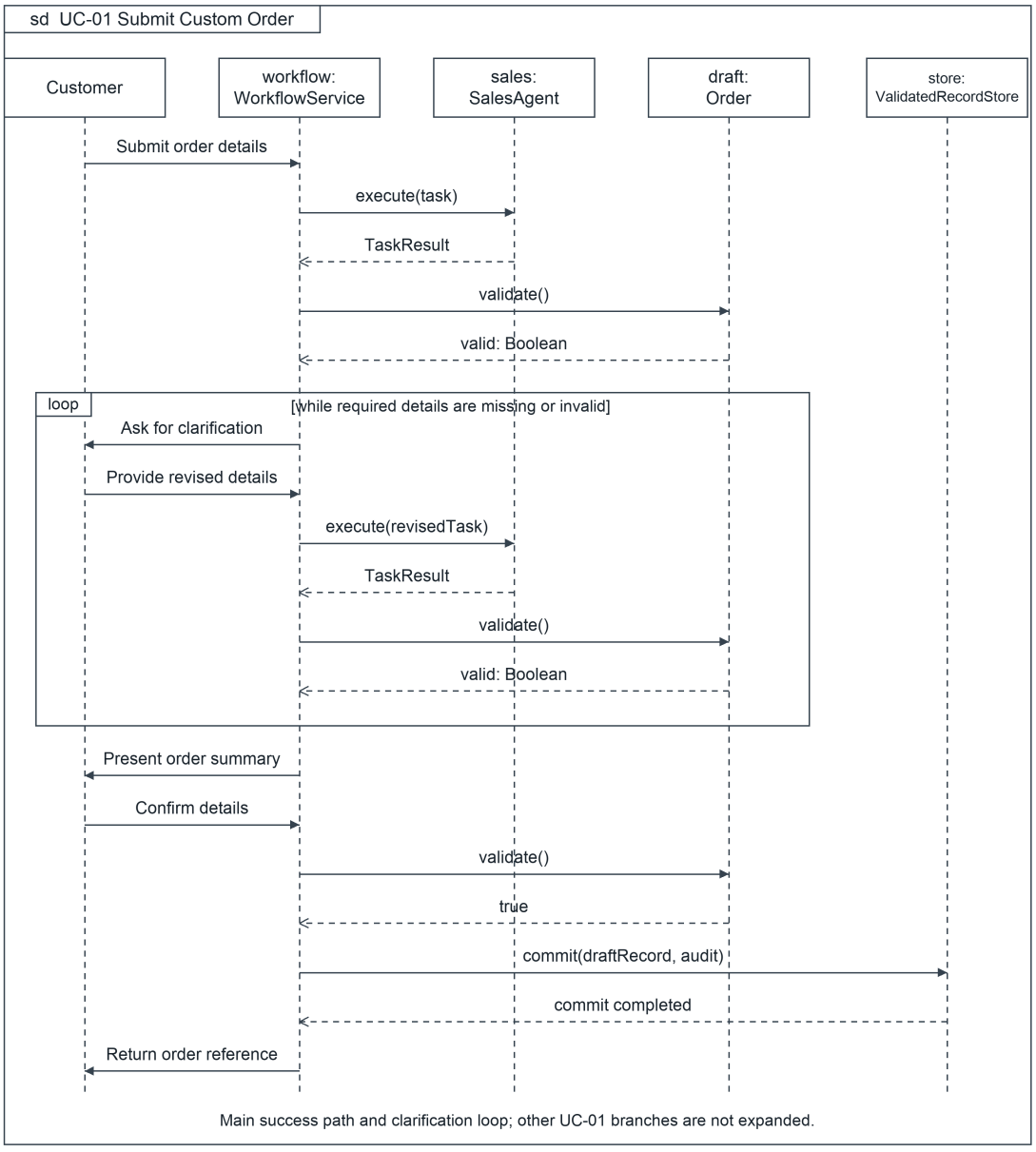


Figure 15: Sequence diagram for order submission, clarification, customer confirmation and draft saving.

Solid arrows represent calls or user requests; dashed arrows represent returns. The `Order` lifeline is an in-memory draft populated from the agent's result, not a saved

record. Internal LLM calls, authentication, owner review and other alternative or failure paths are omitted for readability and remain specified in UC-01. Any required owner review must be resolved before confirmation and saving. An order reference is returned only after a successful save; the sale is not yet formally confirmed.

11.2 Inventory and Production

Figures 16 and 17 form one UC-04 sequence for AR-02. Part 1 obtains and validates a proposal; Part 2 continues with the same schedule/version for approval and persistence. One candidate order is shown, with multi-order commitments handled under IP-02. Different participating services are expanded on each page for readability.

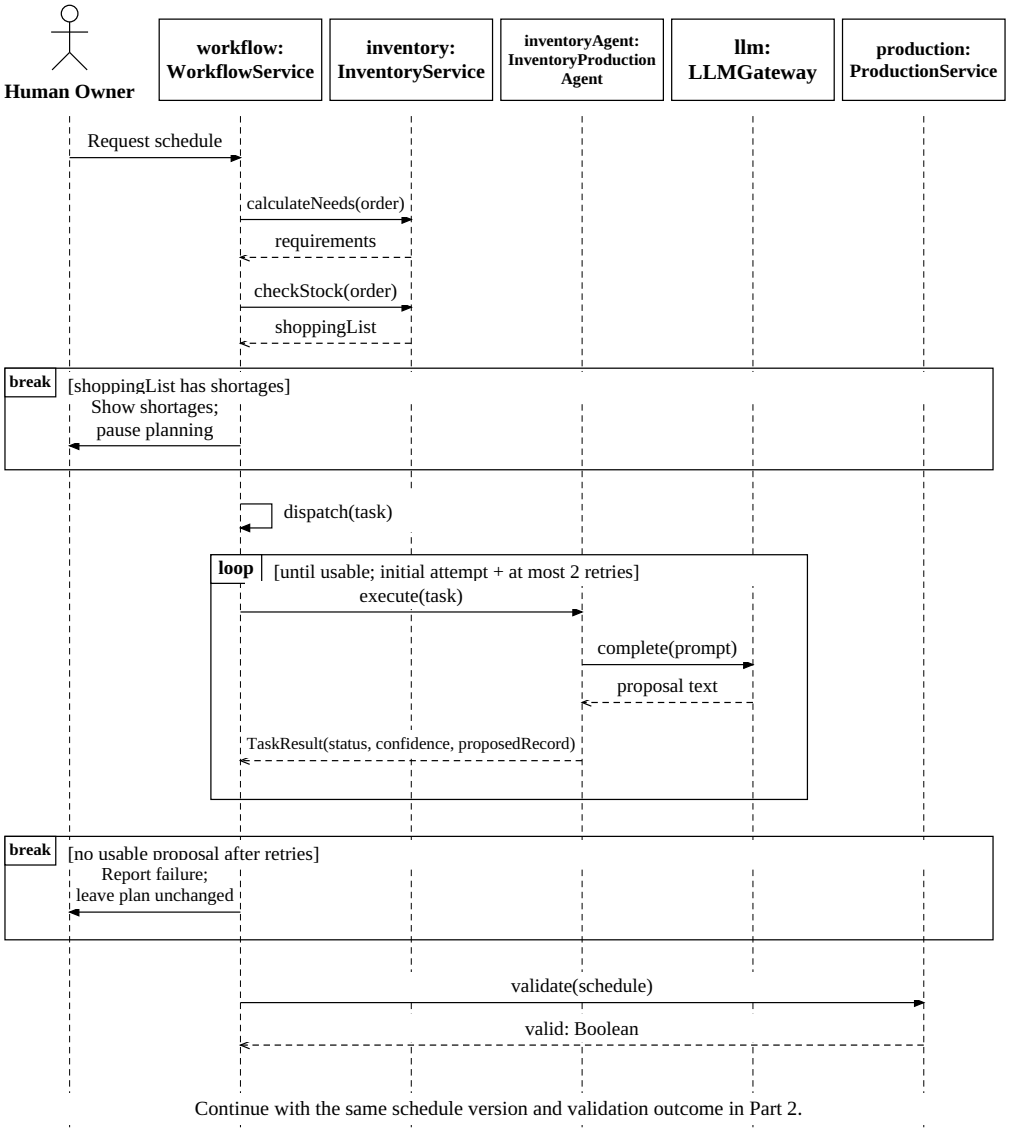


Figure 16: UC-04 sequence, Part 1: ingredient checking, bounded AI retries and schedule validation.

The existing TaskRequest carries the order, calculated requirements, stock result and planning constraints. The agent returns a proposal, not permission to write business

records. Solid arrows show calls or user requests; dashed arrows show returns. User requests and the revision self-message describe workflow behaviour rather than new class operations. Routine authentication and internal record reads are omitted.

Part 2 records a decision for the current version, rechecks the plan and performs a guarded save. Revision returns to Part 1. The messages use the existing Record and AuditEntry supporting types.

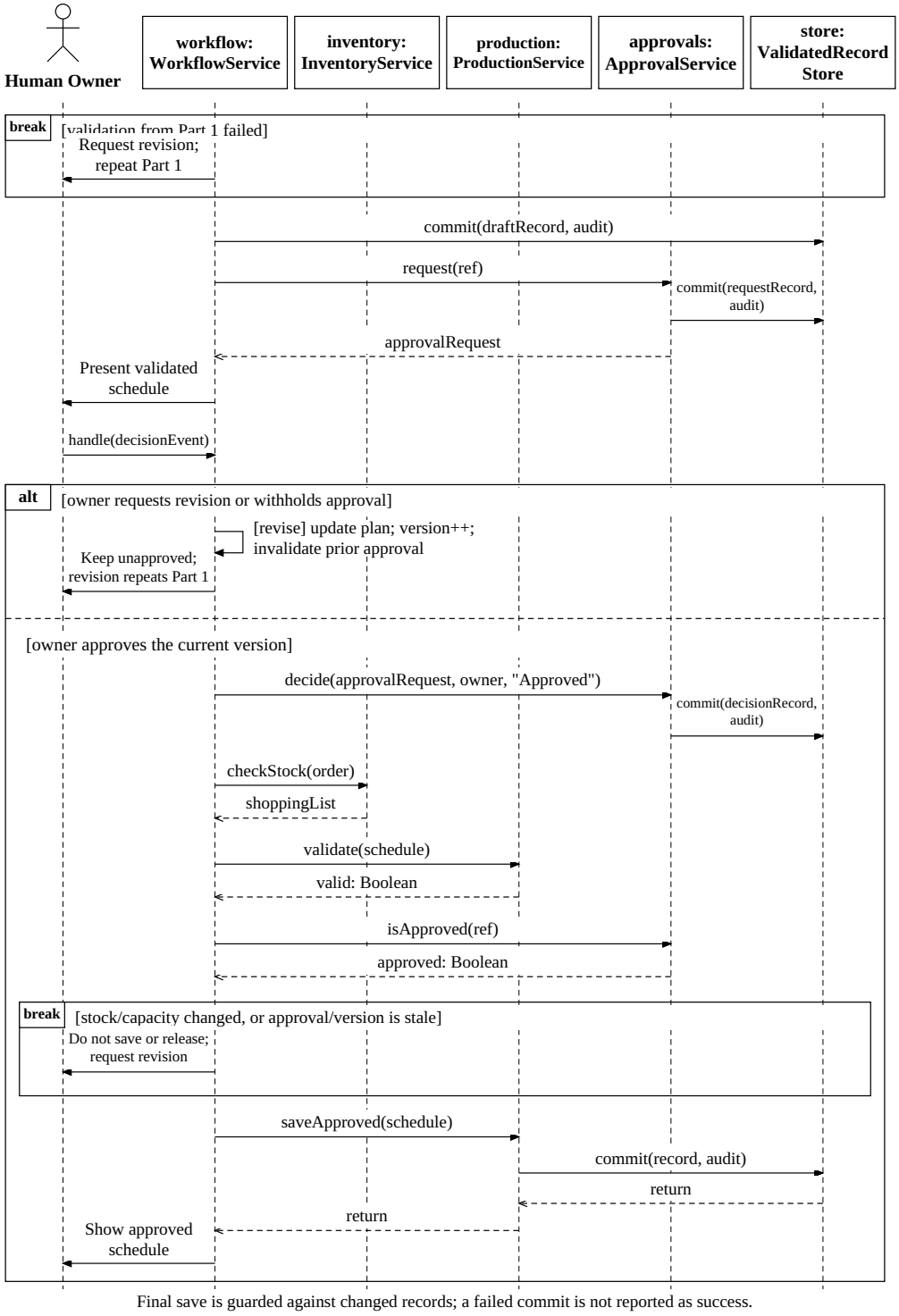


Figure 17: UC-04 sequence, Part 2: owner approval, fresh checks and guarded persistence.

11.3 Pricing and Bookkeeping

Figures 18 and 19 form one UC-05 sequence for AR-03, *Generate and Approve Quotation*. Part 1 covers calculation, explanation and the owner decision; Part 2 continues with the same approved quotation version for the customer decision and order confirmation.

In Part 1, the Workflow Service invokes `FinanceService`, which retrieves the costing records and calculates the quotation deterministically. The Pricing and Bookkeeping Agent then calls the external language model through `LLMGateway` to explain the calculated price. The LLM is used only for explanation: its output is returned to the Workflow Service and checked against the calculated breakdown, with at most two retries, and it cannot change any financial value. The draft quotation and its approval request are stored before owner review, and low-profit, unusually high or unexplained quotations are flagged for manual review. A revision creates a new quotation version, so the outer loop repeats calculation, explanation and renewed approval.

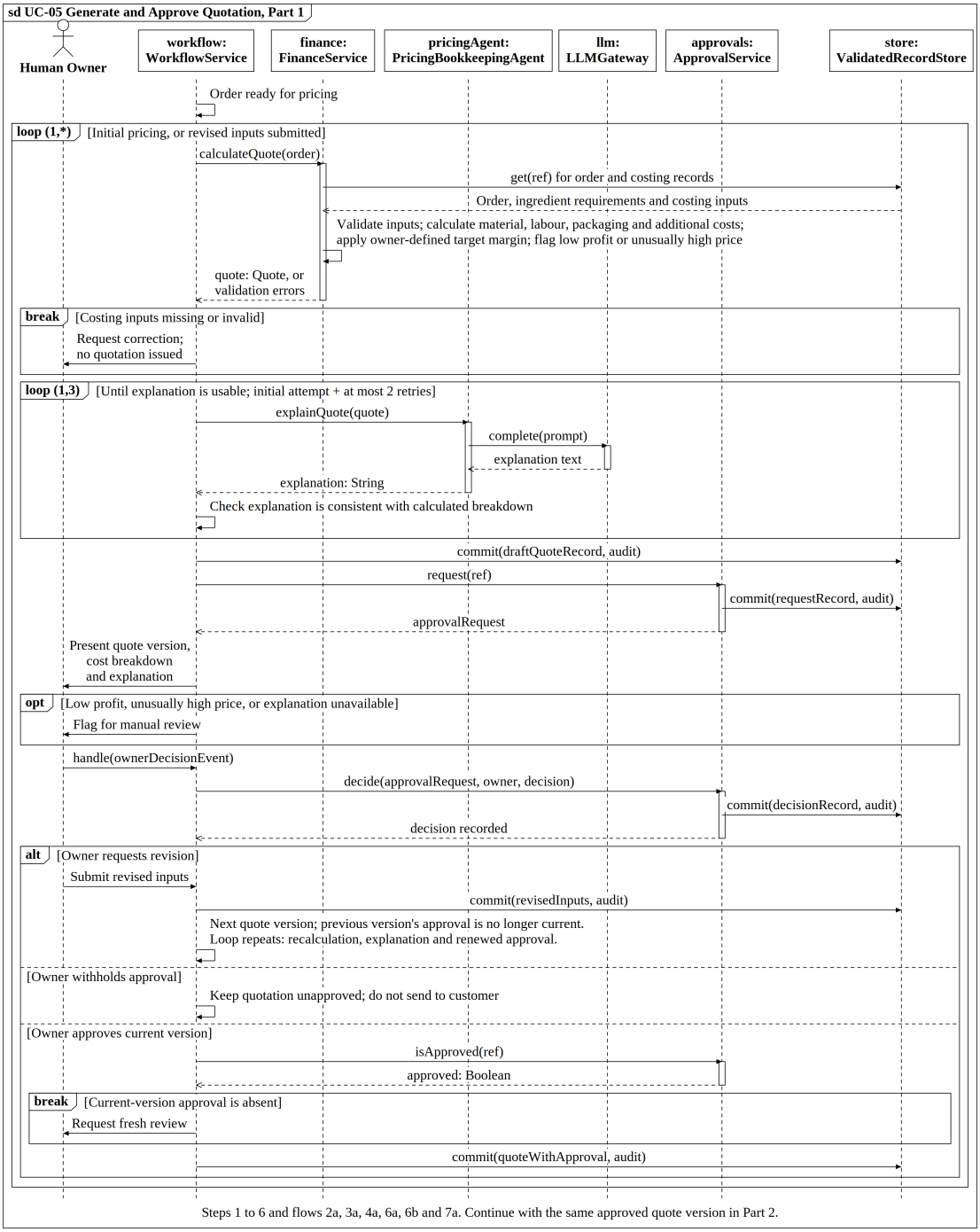


Figure 18: UC-05 sequence, Part 1: deterministic quotation calculation, LLM-supported explanation and owner decision.

In Part 2, the approved quotation is sent to the customer. On acceptance, the system re-verifies the version and its approval before `OrderService` confirms the order. A rejection is recorded and the owner is informed; if the owner proceeds with revised terms, the sequence returns to Part 1. Solid arrows represent calls or user requests, dashed arrows represent returns, and routine authentication is omitted.

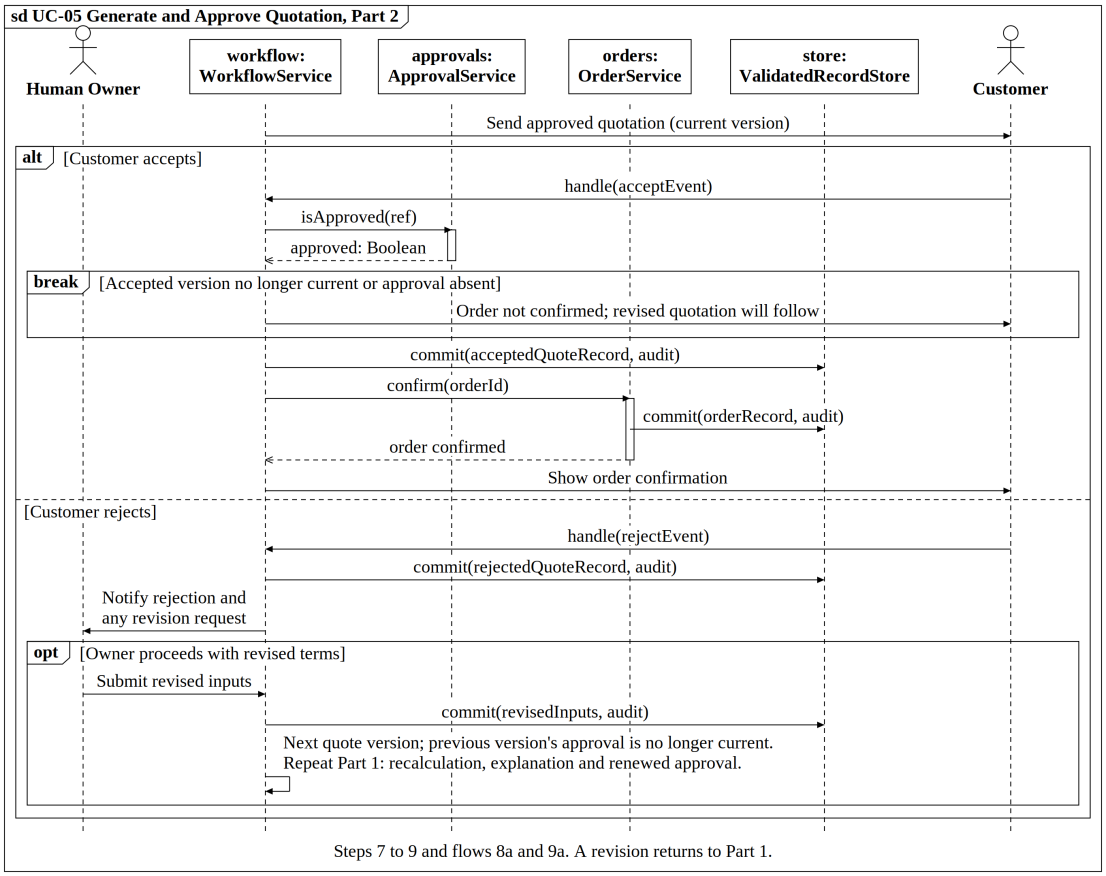


Figure 19: UC-05 sequence, Part 2: customer decision, approval re-verification and order confirmation.

11.4 Marketing

Figure 20 models the interactions for UC-08, *Generate and Approve Marketing Content*, and realises requirement AR-04. The Workflow Service coordinates content generation by the Marketing Agent, validation by the deterministic MarketingService, and the owner approval recorded through ApprovalService.

The Marketing Agent builds a prompt from the approved campaign plan, calls the external language model through LLMGateway, and interprets the response into a proposed MarketingContent record. The proposal is returned to the Workflow Service, which never allows the agent to write to the record store directly. MarketingService then validates the referenced products, prices and availability against stored records. A failed check corresponds to a false result from validate(); the diagram shows the resulting error list so that the reported errors are explicit. The optional fragment shows a retry when errors are returned. A retried proposal passes through the same validation before an approval request is created, and if no valid proposal is obtained after the initial attempt and two retries, the failure is reported to the owner and no approval request or content record is created. These repeated checks and the escalation path are specified in UC-08 and are not expanded in the diagram.

Once validated, one approval request of kind PublicContent is created for the current content version and the draft is presented to the owner. This is the same request

created on entry to AwaitingOwnerDecision in Figure 24, not a second creation step. For readability, the owner's decision is shown as a direct message to ApprovalService. At runtime it is submitted through the owner dashboard and handled by the Workflow Service, which forwards it to ApprovalService, as in Figure 18. The alternative fragment shows the two outcomes of the owner decision. An approval is recorded and the approved content is prepared for manual export; a rejection is recorded and a revision is requested from the Marketing Agent. In both cases the outcome is reported to the owner. ApprovalService commits each decision with an audit entry through ValidatedRecordStore, whose lifeline is omitted here; the same commit pattern appears in Figure 18. Messages such as requestContent, presentForApproval, retry, requestRevision and notifyOutcome describe workflow interactions rather than additional class operations. Approval and export checks apply to the current content version. Later edits, including edits to content already in the calendar, follow UC-08 flow 8a and the revision path in Figure 24.

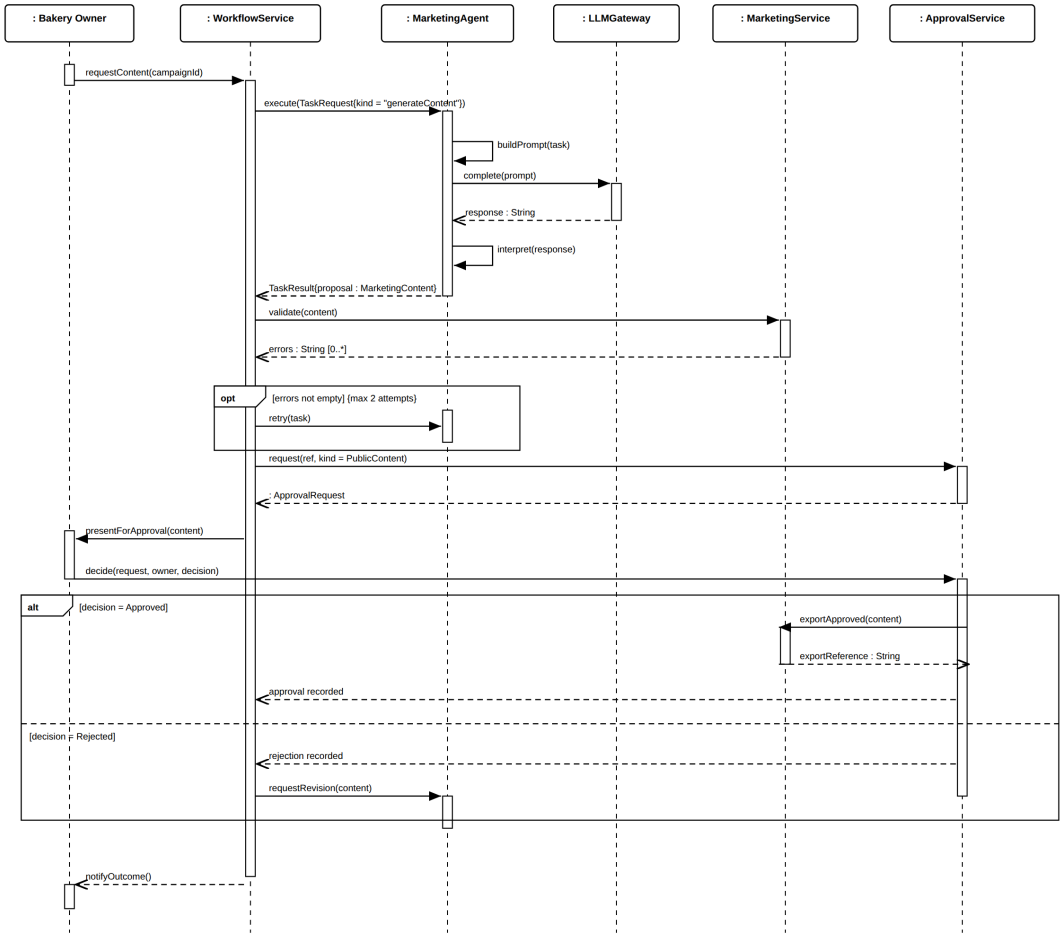


Figure 20: Sequence diagram for marketing content generation, deterministic validation, retry on failure, and owner approval or rejection.

12 State Machine Diagrams

12.1 Sales and Customer Service

Figure 21 models the submission lifecycle for UC-01, *Submit Custom Order*, and supports AR-01. The request moves from collecting details to validation and customer confirmation. Missing or invalid information returns for clarification. Requests involving ingredient customisations or uncertain allergen information wait for owner review before being checked again.

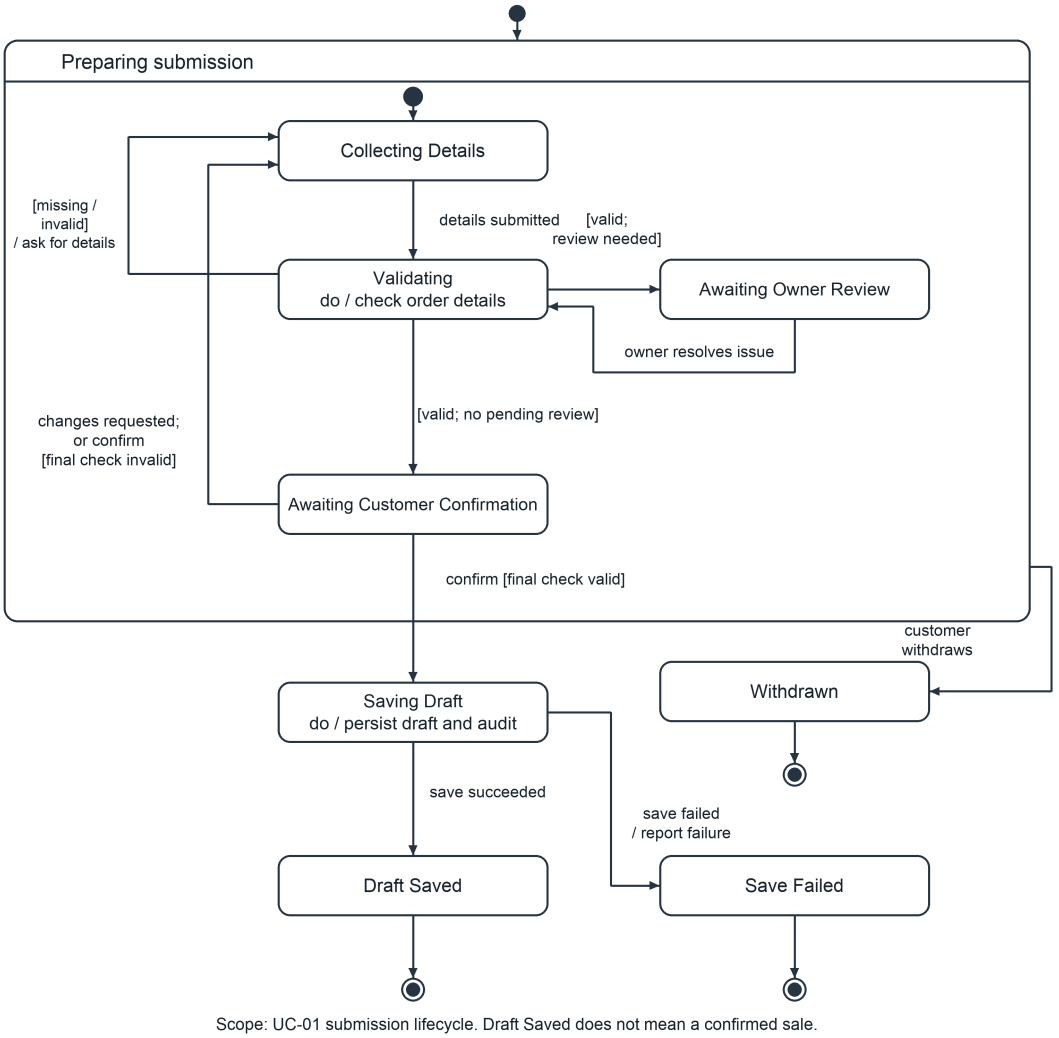


Figure 21: State machine for order submission, review, customer confirmation and draft saving.

The composite state *Preparing submission* groups the stages before persistence begins. Its withdrawal transition allows the customer to end the request from any of these stages. Customer corrections or a failed final check return the request to detail collection. A confirmed request enters *Saving Draft* only after the final check succeeds. The save outcome leads to *Draft Saved* or *Save Failed*.

These states describe the Workflow Service's handling of one submission, rather

than additional values of the class model's `Order.status` attribute. Final nodes end this submission attempt, not the lifetime of a successfully saved order. AI retry and other failure details remain specified in UC-01. A saved draft still requires quotation and customer acceptance before the sale is formally confirmed; no inventory is deducted by this state machine.

12.2 Inventory and Production

Figure 22 models ProductionSchedule preparation and approval for UC-04 and AR-02. States describe existing workflow, version and approval conditions, not a new schedule status attribute. Validation precedes owner review. Approval triggers fresh checks and a guarded save; only success reaches *Approved*.

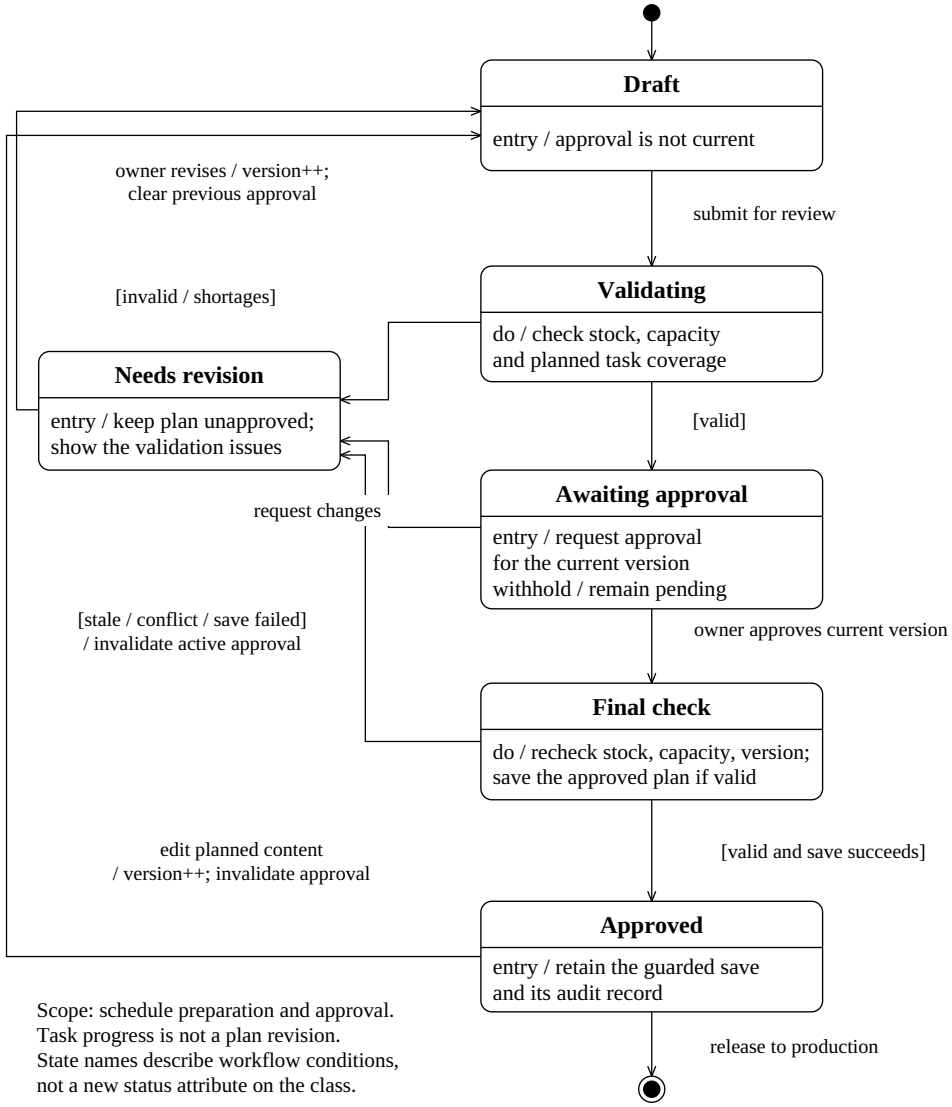


Figure 22: ProductionSchedule approval lifecycle: validation, revision, version-specific approval and release to production.

Withholding approval leaves the schedule pending. Editing planned content creates

a new version requiring fresh approval. The final node ends this approval workflow on release, not the order's production or the schedule's existence. ProductionTask records execution progress, which does not itself revise the plan. IP-01 still requires completion of all required tasks before once-only order deduction.

12.3 Pricing and Bookkeeping

Figure 23 models the lifecycle of a `Quote` for AR-03 and UC-05, *Generate and Approve Quotation*. Its states are conditions of the existing `Quote` attributes `version`, `approval` and `customerDecision`, not values of a new status attribute, following the approach of Figure 22. The `orderReadyForPricing` event starts the lifecycle, and `FinanceService` calculates the quotation deterministically. Invalid costing inputs move the quote to Awaiting input correction until the owner corrects them (2a). The Pricing and Bookkeeping Agent then explains the result. If the explanation still fails after two retries, the quote enters Manual review directly (4a), as do low-profit or unusually high quotations (3a). Withholding approval leaves the quote unapproved (6b), while a revision request returns it for recalculation as a new version and clears the previous approval (6a). Only approval of the current version sends the quote to the customer (7a). If the customer accepts a version that is no longer current or approved, the order is not confirmed and the quote returns to owner review (9a). A rejected quote is not terminal: the owner can request a revision and repeat the cycle (8a), or close the quote.

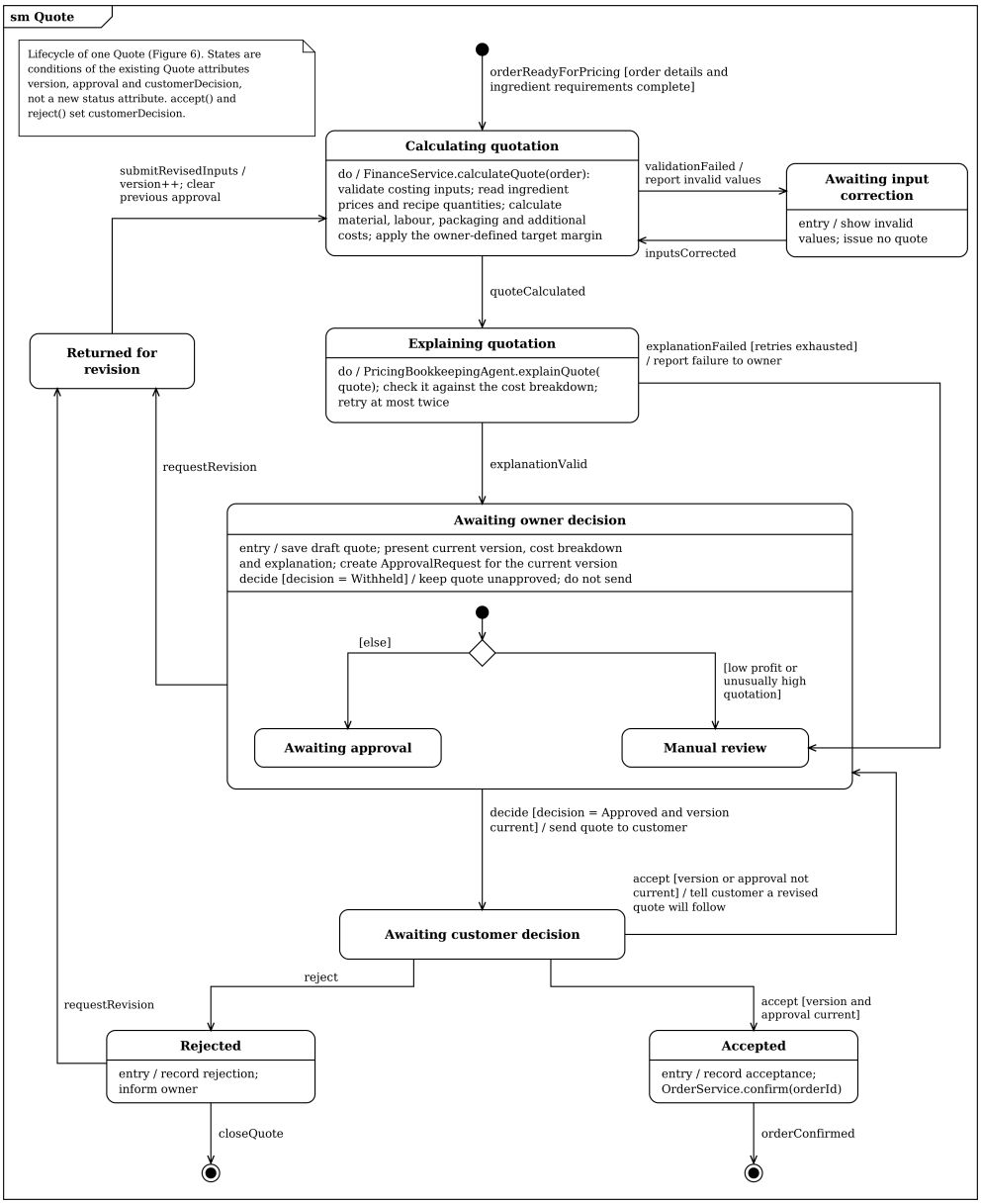


Figure 23: Quote state machine showing deterministic pricing, input correction, explanation failure and manual review, version-specific owner approval and revision, and customer acceptance or rejection with a return to owner review for stale versions.

12.4 Marketing

Figure 24 models the lifecycle of a `MarketingContent` record for AR-04 and UC-08, *Generate and Approve Marketing Content*. The states describe conditions of its existing `version` and `approval`, together with validation and `isReadyForExport()` results; they do not introduce a new status attribute. Transition labels such as submitForValidation and addToCalendar describe workflow events rather than additional class operations.

Content starts in Draft after generation from an approved campaign. The Validating state checks referenced products, prices and availability through `MarketingService`.

Failure reports the errors and returns the content to Draft. Success enters AwaitingOwnerDecision. Entry to that state creates one `ApprovalRequest` of kind PublicContent for the current content version; the validation-success transition does not create another request. Failed AI generation and retries follow UC-08 and are omitted from this record-lifecycle view.

A current-version owner approval moves content to Approved. Rejection moves it to Rejected, from which revision can restart the cycle. Requesting a different version also returns content to Draft and increments its version. Only content that passes `isReadyForExport()` can move from Approved to Scheduled. Scheduled means that the approved version is in the content calendar and is available for the owner to export and publish manually; it is not automatic publication or an immediate terminal state.

Approved and Scheduled are grouped within the composite state *Approved version*. Its outgoing revision transition applies in either state: editing increments the version, invalidates the old approval and returns the content to Draft. Validation and owner review must then be repeated before the revised version can be exported. This covers UC-08 flow 8a for content that has already been scheduled.

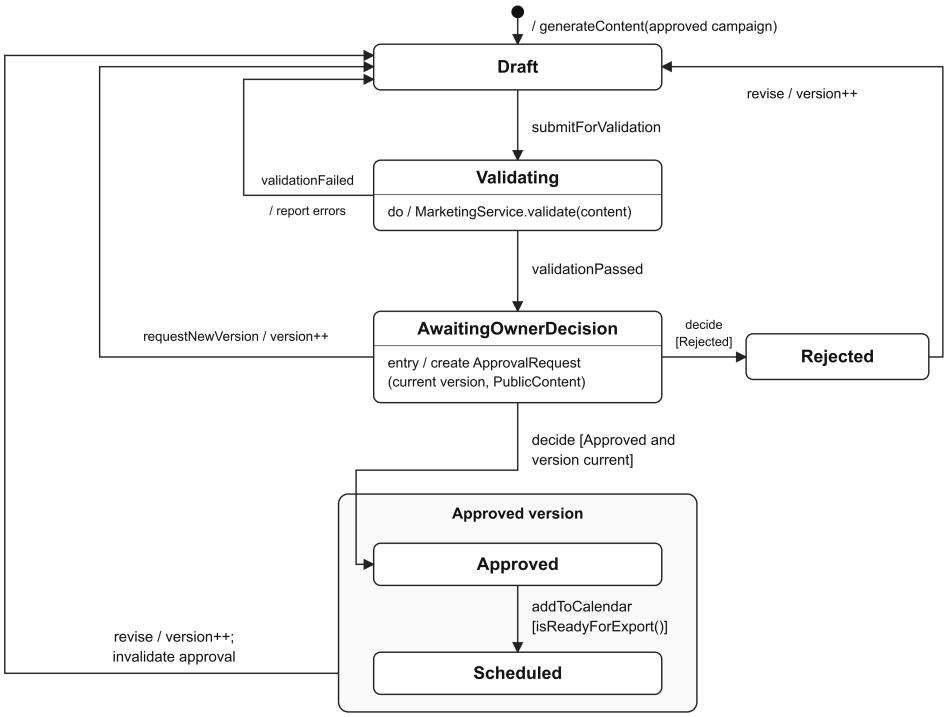


Figure 24: Marketing content lifecycle with one approval request on entry to owner review, version-specific approval, and renewed validation and review when Approved or Scheduled content is edited. Publication remains manual.

13 Package Diagram

Figure 25 groups the existing class model into four business packages and two supporting packages. Each business package contains its specialised agent and corresponding deterministic services. The enclosing Business Modules package groups Sales and Customer Service, Inventory and Production, Pricing and Bookkeeping, and Marketing.

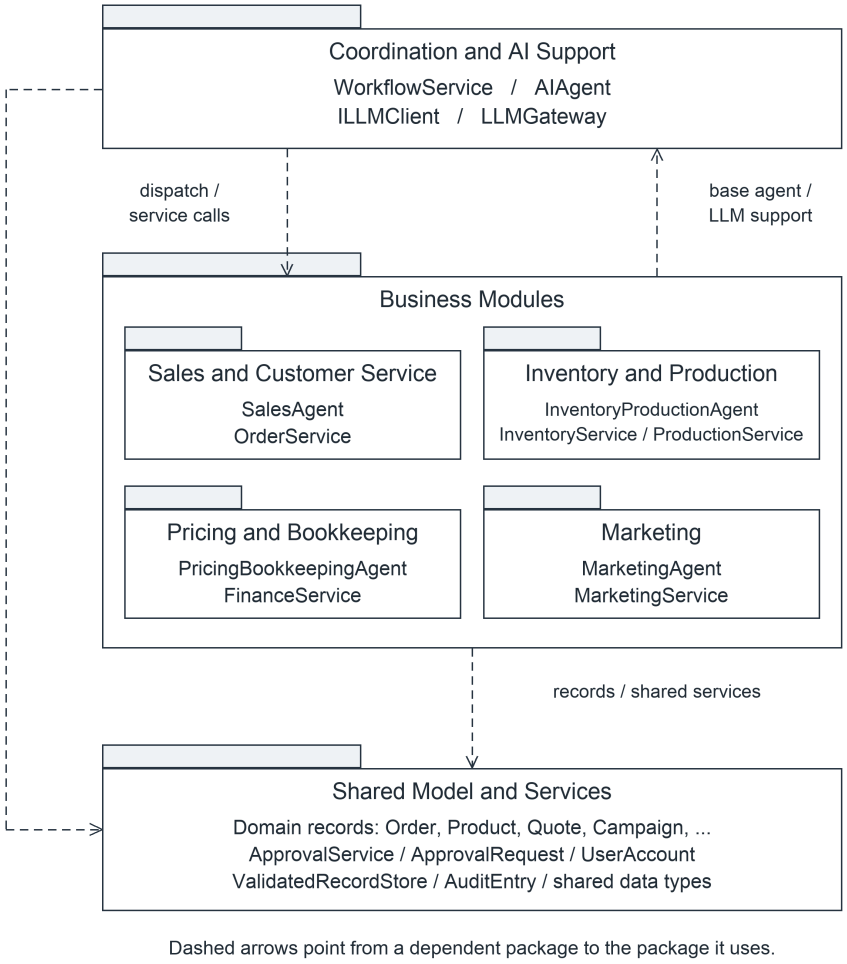


Figure 25: BakeFlow AI package organisation and main dependencies.

Coordination and AI Support contains the Workflow Service, the common agent base class and the LLM interface and gateway. It depends on the business packages to dispatch tasks and invoke services; the specialised agents depend on the common agent and LLM support. Shared Model and Services groups the domain records, approval, access control and validated persistence facilities used across the system.

Dashed arrows point towards the package being used. Dependencies involving Business Modules summarise those of its contained packages, and only representative contents are shown. This is a logical organisation of the existing model; it does not introduce additional business modules or imply separate deployment units.

14 Deployment Diagram

Figure 26 shows the proposed deployment of BakeFlow AI. Customers and the bakery owner access the system through web browsers, using the customer interface and owner dashboard respectively. Both interfaces communicate with the application server over HTTPS.

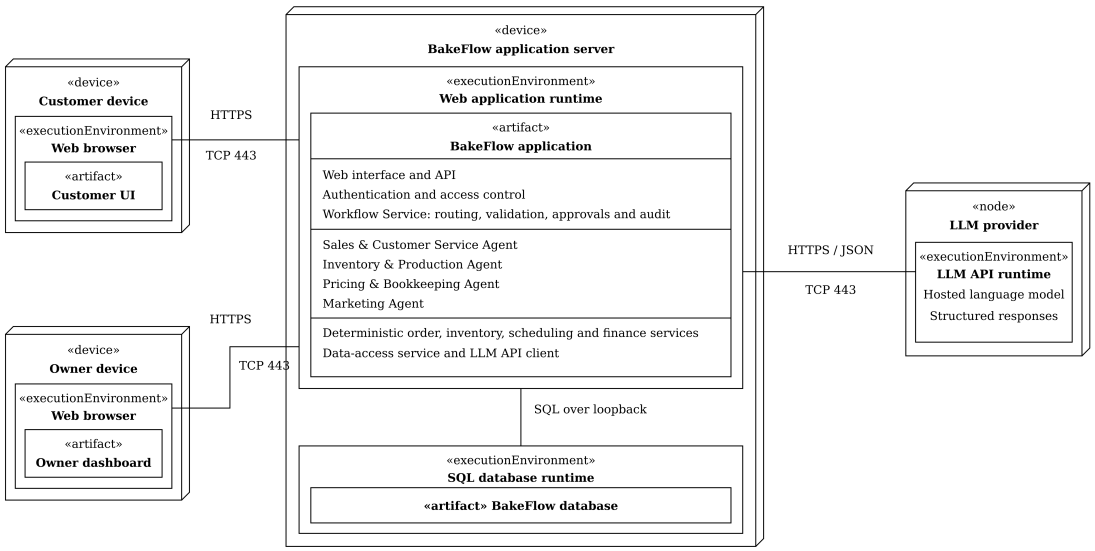


Figure 26: Proposed deployment of BakeFlow AI across client devices, the application server and an external LLM provider.

The application server hosts the four specialist AI agents, the deterministic Workflow Service, business calculation services and the data-access service provided by `ValidatedRecordStore`. These modules run within one application runtime. The Workflow Service manages task routing, validation and owner approvals, while deterministic services perform inventory, scheduling and financial calculations. The agents access the external LLM through a server-side API client for interpretation, planning, explanation and content generation.

The shared SQL database runs on the same server and is accessed locally through the data-access service. Browser clients and the external LLM have no direct database connection. Authentication, access checks and validation are handled by the application before protected operations or persistent updates are permitted.

This deployment assumes a single application host, browser-based interfaces and a remotely hosted LLM. The implementation framework, database engine and LLM provider remain to be selected.

15 Design Assumptions

The Stage 1 design rests on the following assumptions. They are recorded explicitly because several of them bound the scope of the models in this report and would have to be revisited before the system could be used commercially.

Scope of the prototype. The prototype covers custom cakes and cupcakes. Customer, inventory and financial data are simulated rather than drawn from a live business. Orders are collected, but delivery routing and optimisation are out of scope.

External systems. The system does not connect to a real bank or payment provider, and it does not publish content to social media platforms automatically. Approved marketing content is exported for the owner to publish manually. The language model is treated as an external service reached through a single gateway, and no specific provider is assumed.

Business rules supplied by the owner. The owner defines the labour rate and the target profit margin. Pricing therefore depends on owner-supplied parameters rather than on market data, and the system does not attempt to recommend a margin.

Data and calculation. Monetary values use decimal arithmetic to two decimal places. Each product has exactly one recipe, each production schedule owns its tasks, and a transaction is attributed to at most one order and one product; multiple allocations are represented as separate entries. Stock is deducted only once per order, after production is recorded as complete.

Allergen and safety information. Allergen answers are derived only from stored recipe data. Where that data is incomplete or the request is ambiguous, the question is escalated to the owner rather than answered by an agent. The system does not provide tax, legal, medical or food-safety advice.

Reliability of AI output. AI proposals require structural and business validation. Failed, malformed or otherwise unusable output follows the relevant use case: at most two retries after the initial attempt, followed by escalation or an unavailable-explanation notice. Such output does not authorise a business update; independently valid service results are retained as the use case specifies. Low confidence is interpreted in context. UC-07 flow 2a permits a valid general campaign suggestion based on limited data to carry a low-confidence notice into explicit owner review. This exception does not waive business checks or allow automatic approval. Agent explanations never alter the values calculated by deterministic services.

Users and access. A single owner account is assumed. Customers interact with the system through the customer interface, while owner-only functions require authentication. Concurrent editing of the same record by multiple owner sessions is not modelled.

Approval semantics. An approval applies to a specific version of a record. Editing an approved record invalidates that approval and requires the record to be validated and approved again.

16 Group Contributions

All four members contributed to the group discussions that produced the problem statement, the system components, the design decision to replace the Coordinator Agent with a rule-based Workflow Service, and the design principles that follow from it. Each member owns one specialist AI agent, together with the ad hoc requirement, use case specifications and behavioural diagrams for that agent, and the interface between their module and the Workflow Service. Individual contributions are set out below.

Member	Contribution
Ziyi Wang	Sales and Customer Service Agent. Project requirements section; feature diagram; package diagram; ad hoc requirement AR-01; use case specifications UC-01 and UC-02; activity, interaction and state machine diagrams for the sales and customer service workflow; supporting report text for those sections.
Chuanyang Wang	Inventory and Production Agent. Overall use case diagram, with Joshua Bates; complex structure modelling; runtime and object collaboration views; ad hoc requirement AR-02 and the associated modelling and execution rules IP-01 to IP-05; use case specifications UC-03 and UC-04; activity, interaction and state machine diagrams for the inventory and production workflow; supporting report text for those sections.
Joshua Bates	Pricing and Bookkeeping Agent. Overall use case diagram, with Chuanyang Wang; elementary structure modelling and the design-time class diagram; deployment diagram; ad hoc requirement AR-03; use case specifications UC-05 and UC-06; activity, interaction and state machine diagrams for the quotation and bookkeeping workflow; report assembly; supporting report text for those sections.
Angelina	Marketing Agent. Ad hoc overall software design; architecture analysis and design, including the selection of architectural patterns and the 4+1 viewpoints; design assumptions; group contributions; ad hoc requirement AR-04; use case specifications UC-07 and UC-08; activity, interaction and state machine diagrams for the marketing workflow; supporting report text for those sections.

Table 2: Individual contributions to Stage 1.

All group tasks (feature diagram, use case diagram, class diagrams, complex structure, architecture, package and deployment diagrams) were contributed to equally by all four members (25% each); the members named above led the drafting of each item. All members reviewed the shared diagrams for consistency between the requirements, use cases, architecture and structural models, and reviewed the complete report before submission.